# Informe de base de datos, API propia en PHP y hostings de Paglot

Fecha: 19-10-2026 · Ámbito: estado de la nube actual (Supabase), propuesta de API propia en PHP 8.4 con MariaDB, comparativa de hostings, correo y dominios, y migración sin pérdidas.

**Cómo leer este informe**

- **Verificado**: dato leído hoy en la web oficial del proveedor (consultada el 19-10-2026).
- **Resultado de búsqueda**: dato de una fuente secundaria o de un resultado de búsqueda que no se pudo abrir en la web oficial. Hay que comprobarlo antes de decidir.
- **Estimación** («estimación»): cálculo o cifra no confirmada.
- Precios en euros sin IVA, salvo que se indique. El IVA general en España es el 21 %.
- El código de los capítulos 5 y 7 **no se ha ejecutado**: en este equipo no hay PHP. Antes de usarlo en serio hay que pasarlo por las pruebas del capítulo 6.
- **No se ha modificado nada del repositorio.** Solo se han leído archivos. Los ficheros de código de ejemplo están en la carpeta de trabajo temporal, no en el proyecto.

## Índice


- 1. Resumen ejecutivo
- 2. Auditoría del estado actual
- 3. Arquitectura propuesta con PHP
- 4. Esquema SQL completo (MariaDB)
- 5. Código PHP de ejemplo
- 6. Cómo probarlo en local en Windows
- 7. Migración desde Supabase: exportar, transformar, importar
- 8. Hostings, correo, dominios y recomendación
- 9. Seguridad y operación
- 10. Checklist paso a paso para Javier
- Fuentes


## 1. Resumen ejecutivo

**Estado actual.** Paglot guarda el progreso en el dispositivo y, si hay cuenta, lo sube a Supabase (`js/nube.js`, `js/config.js`, `_herramientas/supabase-cuentas.sql`). La cuenta es un número de 16 cifras que genera el propio móvil o navegador. El correo es opcional: sirve para que la cuenta no caduque y para recuperar el número. El SQL de errores e ideas (`_herramientas/supabase-errores-e-ideas.sql`) está escrito pero **no aplicado**; el formulario de contacto **no envía nada**; el laboratorio de ideas guarda todo en el navegador.

**Hallazgos más graves** (detalle en el capítulo 2):

1. **El número es la única llave y no tiene límite de intentos.** Con la clave pública, cualquiera puede llamar a `paglot_leer`, `paglot_existe` y `paglot_correo` sin límite. Un número acertado da el nombre y todo el progreso. Con 1.000.000 de cuentas y 1.000 peticiones por segundo, el primer acierto llega de media en unos 104 días (estimación).
2. **Cualquiera con un número puede quitar la recuperación de una cuenta ajena.** `paglot_vincular` no comprueba si la cuenta ya tiene correo: quien conoce el número une su propio correo verificado y la persona dueña pierde la forma de recuperarla.
3. **Pérdida de datos entre dispositivos.** Al guardar, la app escribe el curso actual en la nube con la copia local, sin juntarla antes con la de la nube. Si se estudia en dos dispositivos, lo de uno se pisa con lo del otro.
4. **Bloqueo para todos.** Hay cuotas globales (500 cuentas nuevas cada 10 minutos, 300 errores cada 10 minutos, 60 ideas por hora). Un atacante anónimo puede agotarlas y dejar sin registros, errores e ideas a todos.
5. **El aviso legal no coincide con la app.** Habla de «contraseña cifrada» (ya no existe), dice que sin cuenta «nada sale del dispositivo» (los errores sí salen), que los datos «se conservan mientras tengas la cuenta» (las cuentas anónimas se borran a los 7 días) y que no se usa la IP (el proveedor la recibe en cada petición). El responsable sigue sin rellenar (`TITULAR` vacío en `config.js`). Hay que corregirlo antes de publicar.
6. **La supresión no es completa.** Al borrar la cuenta desaparece la fila de progreso, pero el usuario de `auth.users` con su correo queda. Además, el código de un solo uso crea usuarios para correos que nunca han usado la app.
7. **Material antiguo con secretos.** En `Paglot_material_antiguo\…\servidor\config.php` hay contraseñas de base de datos y una clave de Resend. Este informe no las ha copiado ni citado. Conviene retirarlos de esa carpeta y cambiarlos si alguna vez se compartieron.

**Recomendación** (capítulos 3 a 9):

- Mantener el modelo de «número de cuenta» para el usuario, pero **el número deja de ser la llave de cada petición**. Entrar en otro dispositivo cambia el número por un **token de dispositivo** revocable. El servidor guarda el número solo como HMAC con un secreto.
- API propia en **PHP 8.4** (soporte de seguridad hasta el 31-12-2028; soporte activo hasta el 31-12-2026, según una fuente secundaria) con **MariaDB 11.8 LTS**, Nginx y PHP-FPM, en un **VPS de la UE**. El precio orientativo de Hetzner (Alemania o Finlandia) es de unos 6 € al mes sin IVA (estimación).
- Web estática en **Cloudflare Pages** (plan gratis, verificado) y **Cloudflare Turnstile** en el formulario (gratis según Cloudflare; el texto consultado no lo dice expresamente, hay que comprobarlo).
- Correo: **Resend** en el plan gratis (3.000 al mes, máximo 100 al día; verificado) para empezar. Cuando los accesos por correo superen los 100 al día, plan Pro (20 $ al mes, verificado) u otro proveedor de la UE.
- Coste orientativo: unos **12 € al mes** en infraestructura sin correo de pago, y unos **30 €** con Resend Pro (estimación). Un hosting compartido de 3 a 9 € al mes serviría, pero hoy **no está confirmado** que tenga SSH, cron y PHP 8.3 o superior (capítulo 8).
- Migración sin pérdidas: congelar las escrituras en Supabase, exportar, importar con los **mismos números** y mantener un **adaptador** que hable el protocolo antiguo, porque las APK ya instaladas siguen llamando a Supabase (capítulo 7).

**Lo que este informe no hace.** No ejecuta PHP (no hay en el equipo), no toca el repositorio y deja marcadas como estimación las cifras que no se han podido confirmar.

## 2. Auditoría del estado actual

Se han leído: `js/nube.js`, `js/config.js`, `js/progress.js`, `js/vistas/perfil.js`, `js/vistas/contacto.js`, `js/vistas/laboratorio.js`, `js/curso.js`, `js/cursos.js`, `js/errores.js`, `js/i18n/es.js` (aviso legal), `index.html` (CSP), `_apk/puente.js`, `_apk/capacitor.config.json`, `_apk/copiar.mjs`, `_herramientas/supabase-cuentas.sql` y `_herramientas/supabase-errores-e-ideas.sql`. Del material antiguo, `servidor/` (API PHP de 17-10-2026, descrita en el apartado 2.8).

### 2.1 Qué datos hay y dónde

**En el dispositivo** (`localStorage`):

| Clave | Contenido | ¿Sube a la nube? |
|---|---|---|
| `paglot-cuenta` | Número de cuenta de 16 cifras | Sí, como llave de cada petición |
| `<prefijo>-datos` (21 cursos) | Perfil (nombre, máx. 40 caracteres), lecciones, actividad diaria, ejercicios, racha | Sí, en la copia del curso |
| `<prefijo>-tarjeta:<id>` | Cada palabra o frase del cuaderno: traducción, pronunciación, nota, origen, datos de repaso | Sí, en la copia del curso |
| `<prefijo>-juego2`, `-estrellas`, `-cofres`, `-logros` | Juego, estrellas, cofres y logros | Sí (van como «extras») |
| `<prefijo>-dia-completo`, `-camino-abiertos`, `-prueba`, `-reto-mejor` | Racha del día, camino, lección de inicio tras la prueba de nivel, récords | No |
| `idiomas-curso`, `idiomas-ajustes`, `idiomas-tutora-oculta`, `idiomas-tutora-hoy`, `idiomas-modo-leccion`, `idiomas-aviso-visto`, `paglot-bienvenida-paso` | Curso elegido, ajustes (idioma, tema, sonido, voz, velocidad, recordatorio), preferencias | No |
| `paglot-laboratorio-ideas-v1`, `paglot-laboratorio-votos-v1` | Ideas y votos del laboratorio | No (todavía no hay API) |
| `paglot-sin-nube` | Desactiva la nube (lo usan las pruebas automáticas) | No |
| `espanol-datos` | Datos de la app de solo español (solo se lee, para migrar) | No |

En el APK, además, hay un aviso de notificación de bienvenida (`paglot_welcome_notification_v5`) y la hora del recordatorio diario. No salen del dispositivo.

**En la nube** (Supabase, esquema `public`):

| Objeto | Columnas o contenido | Observaciones |
|---|---|---|
| `paglot_cuentas` | `numero` (text, clave primaria, 16 dígitos **en claro**); `datos` (jsonb, `{version: 3, cursos: {<curso>: copia}}`); `usuario` (uuid → `auth.users`); `correo` (text **en claro**); `creada`, `usada`, `escrita` | RLS activado sin políticas; `anon` y `authenticated` no tienen acceso directo a la tabla |
| `auth.users` (Supabase Auth) | Correo, fecha de creación, último acceso | Lo crea el código de un solo uso para **cualquier** correo (`create_user: true`) |
| `paglot_errores` (si se aplica) | `dia`, `mensaje`, `donde`, `pantalla`, `curso`, `version`, `navegador`, `veces`, `primera`, `ultima` | Sin número ni correo. Se borra a los 90 días |
| `paglot_ideas`, `paglot_votos`, `paglot_denuncias` (si se aplican) | Texto (máx. 180), autora (máx. 32), votos, denuncias, `oculta`, `quien` (marca aleatoria del dispositivo), `creada` | `quien` no es una identidad real: cualquiera puede inventarla |

La copia que sube a la nube es **todo el curso**: perfil con el nombre, cuaderno con las palabras y su origen, y actividad diaria con fechas. Es información personal en el sentido del RGPD, aunque no haya contraseña.

### 2.2 Funciones y permisos

| Función | Quién puede llamarla | Qué hace | Observación |
|---|---|---|---|
| `paglot_leer(numero)` | `anon` | Devuelve todo el progreso de la cuenta; pone `usada = now()`; limpia las cuentas caducadas | Sin límite. Cada lectura ejecuta un `DELETE` sin índice sobre `usada` |
| `paglot_guardar(numero, datos)` | `anon` | Escribe el progreso (crea la cuenta si no existe) | No comprueba la forma del JSON, solo el tamaño. No usa versión: sobrescribe |
| `paglot_existe(numero)` | `anon` | Devuelve si existe una cuenta con ese número | Sirve de oráculo para comprobar números |
| `paglot_correo(numero)` | `anon` | Devuelve el correo unido a la cuenta, **completo** | Quien conoce el número ve el correo |
| `paglot_borrar(numero)` | `anon` | Borra la cuenta | Sin ninguna prueba más que el número |
| `paglot_vincular(numero)` | `authenticated` | Une la cuenta al correo de la sesión verificada | **No comprueba si la cuenta ya tiene correo** (ver B2) |
| `paglot_mi_numero()` | `authenticated` | Devuelve el número de la cuenta unida al correo | El correo verificado da la llave completa |
| `paglot_limpiar()` | Solo interna | Borra las anónimas sin uso desde hace 7 días | Revocada de `anon` y `authenticated` |
| `paglot_error(jsonb)` | `anon` | Anota un error (o suma uno al del mismo día) | 300 filas nuevas cada 10 min, global |
| `paglot_ideas_ver(quien)` | `anon` | Lista las 200 ideas más votadas | Devuelve la autora (texto libre) |
| `paglot_idea_nueva(quien, texto, autora)` | `anon` | Crea una idea | Límite por marca `quien`, que cualquiera puede inventar |
| `paglot_votar(quien, idea, si)` | `anon` | Pone o quita un voto | Sin límite real: marcas inventadas |
| `paglot_denunciar(quien, idea)` | `anon` | Denuncia; con 3 denuncias oculta la idea | Tres marcas inventadas ocultan cualquier idea |

**Permisos.** Es un buen diseño que las tablas no sean accesibles directamente y que todo pase por funciones `SECURITY DEFINER` con `search_path` fijado. Hay un detalle: PostgreSQL concede `EXECUTE` a `PUBLIC` por defecto, y el SQL solo revoca `paglot_limpiar`. Como `anon` hereda de `PUBLIC`, no abre nada nuevo hoy, pero conviene revocarlo explícitamente en cada función.

## 2.3 Límites y caducidades

| Límite | Valor | Dónde | Efecto práctico |
|---|---|---|---|
| Tamaño de la copia | 2 MB **por cuenta, todos los cursos juntos** | `paglot_guardar` | Un usuario con varios idiomas y mucho cuaderno puede dejar de sincronizar (B8) |
| Escrituras | 1 cada 2 s por cuenta | `paglot_guardar` | La app sube a los 3 s de cada cambio: correcto |
| Cuentas nuevas | 500 cada 10 min, **global** | `paglot_guardar` | Permite bloquear el registro de todos (B4) |
| Errores | 300 filas nuevas cada 10 min, **global**; 90 días | `paglot_error` | Mismo problema (B4) |
| Ideas | 3 por marca al día; 60 por hora, **global**; sin enlaces (filtro parcial) | `paglot_idea_nueva` | Igual que lo anterior (B4, B14) |
| Denuncias | 3 denuncias ocultan la idea | `paglot_denunciar` | Ocultación con marcas inventadas (B6) |
| Cuentas anónimas | Se borran tras 7 días sin `usada` | `paglot_limpiar` | Lo lanza cada lectura. Las lecturas también cuentan como uso |
| Cuentas con correo | No caducan | `paglot_limpiar` | Correcto |
| Espera de la app | 8 s por petición | `nube.js` | Sin conexión, la app lo marca como «sin conexión» |
| Código de un solo uso | 6 cifras; validez y límites de envío **no figuran en el código** | Supabase Auth | Hay que mirarlo en el panel de Supabase (B5) |

## 2.4 Seguridad: qué puede hacer alguien con la clave pública

La clave `sb_publishable_…` está en `js/config.js` y dentro del APK. Por diseño es pública: no es el problema. El problema es lo que permite con las funciones de `anon`:

- Leer **cualquier** cuenta si conoce o adivina su número, con nombre, cuaderno y actividad.
- Sobrescribir o borrar **cualquier** cuenta conociendo el número. El servidor no valida la forma del JSON, así que también puede meter datos inventados.
- Saber si un número existe (`paglot_existe`) y ver el correo unido (`paglot_correo`).
- Crear cuentas, errores e ideas, con el límite global (que se puede agotar).
- Pedir códigos por correo para **cualquier** dirección (`auth/otp` con `create_user: true`). Eso envía correos a terceros y crea filas en `auth.users`.
- Votar y denunciar con marcas inventadas.

**Fuerza bruta contra el número.** El primer dígito va de 1 a 9 y los 15 siguientes de 0 a 9. Hay $9 \times 10^{15}$ números posibles (unos $2^{53}$). Si se prueban números al azar y hay N cuentas reales y R peticiones por segundo, el tiempo medio hasta el primer acierto es $9 \times 10^{15} / (N \times R)$. Hipótesis: adivinar al azar, sin información previa. No cuenta lo que tarde cada petición ni lo que aguante el proyecto.

| Cuentas reales (N) | 100 peticiones/s | 1.000 peticiones/s | 10.000 peticiones/s |
|---|---|---|---|
| 10.000 | 285 años | 28,5 años | 2,85 años |
| 100.000 | 28,5 años | 2,85 años | 104 días |
| 1.000.000 | 2,85 años | 104 días | 10,4 días |

*Cálculo (estimación): los valores son tiempos medios hasta un primer acierto; cada acierto da una cuenta entera.* Mullvad, que también usa números de 16 cifras como única credencial, calcula en su blog (de hace unos nueve años, resultado de búsqueda) unos 45.000 millones de intentos por acierto con 100.000 cuentas activas, cifra coherente con este cálculo.

**Conclusión.** A escala pequeña, la fuerza bruta no es el riesgo principal. El riesgo es que **no hay ningún límite de intentos** y que la consulta de números es barata: a partir de cientos de miles de cuentas, un atacante con un botnet ya tiene acierto en semanas. La protección real llega con límites por IP y por intento fallido, y con el token de dispositivo del capítulo 3.

## 2.5 Tamaño y coste

- Una copia típica (estimación): 100 KB por curso. Con 5.000 cuentas, unos 500 MB, que es todo el espacio del plan gratis de Supabase (**500 MB de base de datos y 50.000 usuarios activos al mes**, verificado en `supabase.com/pricing`).
- Por encima de ese punto hace falta el plan Pro (**25 $ al mes**, con 8 GB, 100.000 usuarios activos al mes y 250 GB de salida; verificado).
- Riesgo de tamaño: el límite de 2 MB es **por cuenta** y no por curso. Quien estudie cinco idiomas con miles de tarjetas puede rebasarlo y dejar de sincronizar sin que la app lo explique bien (B8). Hay que medirlo con datos reales antes de decidir.

## 2.6 RGPD y LOPDGDD

| Tema | Situación actual | Riesgo |
|---|---|---|
| Responsable | `TITULAR` vacío en `js/config.js` (la página dice «pendiente de completar») | Requisito del artículo 13 del RGPD: bloqueante para publicar |
| Base jurídica | El aviso dice «tu consentimiento y la prestación del servicio» | Razonable, pero el texto habla de contraseña que no existe (B9) |
| Minimización | Se guarda el correo en claro y el número en claro | Se puede reducir con HMAC y correo enmascarado (capítulo 4) |
| Supresión (art. 17) | Se borra la fila, pero no el usuario de `auth.users` (B7) | Incumplimiento parcial |
| Portabilidad (art. 20) | Hay descarga de copia en JSON desde el Perfil, pero **no** de lo que hay en la nube | Parcial |
| Encargados (art. 28) | Se cita solo Supabase, y sin contrato firmado a la vista | Hay que tener el DPA de Supabase y los de los demás proveedores |
| Transferencias | **No consta la región** del proyecto de Supabase (la URL no lo indica) | Hay que mirarlo en el panel del proyecto: si está fuera de la UE, hay transferencia internacional |
| Registro de actividades (art. 30) | No existe | Obligatorio en la práctica: el tratamiento no es ocasional |
| Menores (LOPDGDD art. 7) | El aviso dice «menos de 14 años, pide permiso» | Correcto, pero no hay ninguna comprobación: solo es un texto |
| Almacenamiento local (LSSI art. 22.2) | Solo almacenamiento necesario, con aviso | Correcto |
| Errores (`paglot_error`) | Mensaje, lugar, pantalla, curso, versión y navegador. Sin número ni correo | Datos pseudónimos. El mensaje podría llevar texto del usuario (B13) |

## 2.7 Debilidades y arreglos, por gravedad

Gravedad: **A** alta (corregir antes de publicar o de dar más usuarios), **M** media, **B** baja, **I** informativa.

| ID | Gravedad | Problema (dónde) | Arreglo concreto |
|---|---|---|---|
| B1 | A | El número es la llave de cada petición y no hay límite de intentos (`paglot_leer`, `paglot_existe`, `paglot_correo`, `paglot_borrar`) | Con la API propia: token de dispositivo y límites por IP y por intento fallido (`Limites`, `Cuentas::entrar`). En Supabase no hay límite por IP en SQL: la solución de fondo es la API |
| B2 | A | `paglot_vincular` sobrescribe el correo de una cuenta ya unida | Ver el parche SQL de abajo. Con la API: 409 si el correo ya está en otra cuenta |
| B3 | A | `subir()` escribe el curso actual sin fusionarlo con la nube; `visibilitychange` escribe desde una caché antigua | Leer el curso de la nube y fusionarlo antes de escribir, o escribir con versión (`PUT` con `version`, 409 si no coincide) |
| B4 | A | Cuotas globales (cuentas, errores, ideas) que un anónimo puede agotar | Cuotas por IP en la API (Nginx y `Limites`). En Supabase, quitar o reducir las cuotas globales |
| B5 | M | `auth/otp` con `create_user: true` crea usuarios y manda correos a cualquier dirección | En la API propia no hace falta `auth.users`. Mientras tanto: revisar en Supabase Auth los límites de envío y la validez del código |
| B6 | M | Las ideas se ocultan con tres marcas inventadas (`paglot_denunciar`) y los votos se inflan igual | Exigir cuenta para votar y denunciar, y una denuncia por cuenta (ya en la API) |
| B7 | M | Borrar la cuenta no borra `auth.users` | Borrar también el usuario de Auth con la clave de servicio (desde el panel, o en la API propia) |
| B8 | M | Límite de 2 MB por cuenta para todos los cursos | Subir el límite o guardar cada curso en su fila (ya es así en la API). Medir con datos reales |
| B9 | M | El aviso legal no coincide con la app (ver abajo) | Reescribir los apartados de datos, conservación, terceros e IP antes de publicar |
| B10 | M | El formulario de contacto comprueba la suma (a + b) **en el navegador**: no es una barrera | Comprobación en el servidor con Turnstile, límite por IP y campo trampa (`Contacto`) |
| B11 | B | `EXECUTE` a `PUBLIC`; limpieza sin índice | `revoke execute … from public` en cada función; índice parcial para la limpieza (ver parche) |
| B12 | B | `paglot_correo` devuelve el correo completo; `paglot_existe` confirma números | Enmascarar el correo; quitar `paglot_existe` si no hace falta |
| B13 | B | `paglot_error` solo recorta el texto a 300 caracteres | Quitar correos y números largos antes de guardar (`Errores::limpiar`) |
| B14 | B | Filtro de enlaces de ideas incompleto (solo algunos dominios) | Lista más amplia y moderación manual con denuncias |
| B15 | I | Documentación desfasada: `nube.js` y `config.js` citan `Para_lodriga/paglot-cuentas.sql` (el archivo real es `_herramientas/supabase-cuentas.sql`); `progress.js` cita `sync.js` y «la API propia», que ya no existen. La CSP de `index.html` solo permite `connect-src` a Supabase | Corregir los comentarios. La CSP hay que cambiarla al migrar |
| B16 | I | Secretos en el material antiguo (`servidor/config.php`, `servidor/local/.claves`, `servidor/local/pruebas-google-claves.json`) | Sacarlos de esa carpeta, borrarlos si ya no hacen falta, y cambiarlos si alguna vez salieron de tu equipo |

**Parche SQL propuesto para B2** (no aplicado; para revisar):

```
create or replace function public.paglot_vincular(p_numero text)
returns boolean language plpgsql security definer set search_path = public as $$
begin
  if auth.uid() is null then raise exception 'hace falta verificar el correo'; end if;
  -- no robar una cuenta que ya tiene correo de otra persona
  if exists (select 1 from paglot_cuentas
             where numero = p_numero and usuario is not null and usuario <> auth.uid()) then
    raise exception 'esta cuenta ya tiene otro correo';
  end if;
  update paglot_cuentas set usuario = null, correo = null where usuario = auth.uid() and numero <> p_numero;
  update paglot_cuentas set usuario = auth.uid(), correo = auth.jwt() ->> 'email', usada = now() where numero = p_numero;
  return found;
end $$;

-- B11: quitar EXECUTE a PUBLIC y limpiar con índice
revoke execute on all functions in schema public from public;
create index if not exists paglot_cuentas_usada_anonimas on public.paglot_cuentas (usada) where usuario is null;
```

Revocar `EXECUTE` de `PUBLIC` no quita los permisos que ya se dieron explícitamente a `anon` o `authenticated`, pero el cambio es global: hay que probarlo primero en un proyecto de pruebas y comprobar que la app sigue funcionando. Por eso se deja como propuesta.

**Aviso legal frente a la app** (B9): estos apartados de `js/i18n/es.js` ya no se corresponden con el código:

- «**tu contraseña (cifrada: nadie puede leerla)**»: la cuenta actual no tiene contraseña.
- «**Sin cuenta: nada sale de tu dispositivo**»: los errores anónimos sí salen.
- «**Se conservan mientras tengas la cuenta**»: no dice que las cuentas anónimas se borran a los 7 días sin uso, ni que los errores se guardan 90 días.
- «**No usamos tu dirección IP**»: el proveedor la recibe en cada petición. Hay que decirlo, aunque no se use para nada.
- Solo se cita **Supabase** como encargado. Faltan los demás proveedores de la migración (capítulo 8).
- El responsable sigue **sin rellenar**.

**Lo que está bien hecho** (para no perderlo al migrar):

- Las tablas no son accesibles directamente, y todo pasa por funciones con `search_path` fijado.
- El número se genera con `crypto.getRandomValues`, no con `Math.random`.
- Los errores no llevan número de cuenta ni correo.
- Las ideas se pintan sin `innerHTML`; el único `innerHTML` de la app son dibujos fijos del catálogo de cursos. Los datos de la nube no llegan a `innerHTML`.
- La clave que va en el APK es la pública.
- Las cuentas anónimas caducan, así que no se acumulan para siempre.

### 2.8 Material antiguo: la API PHP de 17-10-2026

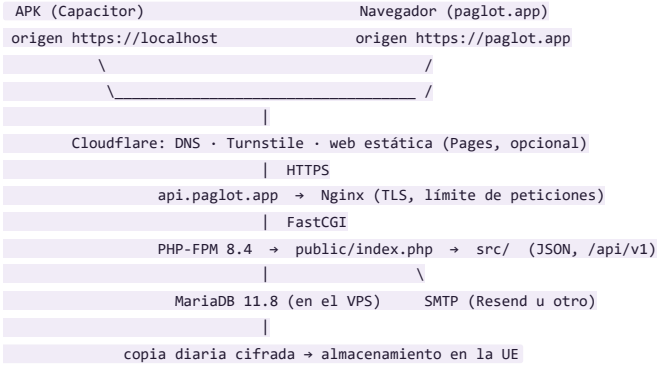
En `Paglot_material_antiguo\Aplicaciones de idiomas desarrolladas - copia\servidor\` hay una **API PHP completa**, probada con 115 pruebas automáticas (`servidor/pruebas/`) y con un `LEEME.md` que describe el diseño. Se dejó de usar, pero su diseño y gran parte del código sirven para la propuesta de los capítulos 3 a 5. Lo reutilizable:

- **Sesiones**: token de 256 bits en `Authorization: Bearer`; en la base solo va su SHA-256; caducidad a 30 días sin uso y 180 días en total.
- **Contraseñas** (solo si algún día vuelven): Argon2id, de 8 a 128 caracteres, rechazo de las comunes y de las filtradas en Have I Been Pwned (con consulta anónima por prefijo).
- **Códigos de 6 cifras**: 10 minutos, se gastan tras 5 fallos, solo vale el último, se guardan con HMAC y hay límite por correo y hora.
- **Límites** con espera creciente y clave en HMAC (`Limites.php`).
- **Progreso con versión**: `PUT` con `version`; 409 con la copia de la nube y, si la copia nueva tiene menos de la mitad de lecciones y palabras, 409 `retroceso` salvo que sea un reinicio. Copia de lo anterior como mucho cada hora, durante 14 días (`Progreso.php`).
- **Enlaces de correo** en `…/#/enlace/…`: el token va detrás de la almohadilla y no llega al servidor.
- **Correos enviados después de responder** (`fastcgi_finish_request`), para que el tiempo de respuesta no delate si una cuenta existe.
- **Usuarios de la base con permisos mínimos** (`bd/permisos.sql`) y un usuario aparte para las tareas.
- **CORS con lista cerrada**, sin `*` y sin reflejar el origen.
- **Foto de perfil** redibujada con GD a 96 px, para quitar metadatos.
- **Un** `LEEME.md` con las rutas y los códigos de error ya traducibles (`datos_invalidos`, `demasiados`, `conflicto`, `retroceso`…).

Lo que **no** conviene reutilizar sin cambios: la unión de cuentas por correo sin contraseña en el mismo esquema, y los archivos `config.php`, `local/.claves` y `pruebas-google-claves.json` (B16).

## 3. Arquitectura propuesta con PHP

### 3.1 Piezas




La web (HTML, JS, cursos) es estática y no cambia. Lo nuevo es solo la **API** y la base de datos.

### 3.2 Elecciones y alternativas

| Parte | Elección | Motivo | Alternativa |
|---|---|---|---|
| Web y APK | Sin cambios de fondo; la web puede ir en Cloudflare Pages o en el mismo VPS | Gratis y rápida | Mismo VPS con Nginx |
| API | PHP 8.4, PDO, sin framework | La puede mantener una persona; sin dependencias salvo PHPMailer; ya hay un diseño probado (apartado 2.8) | Laravel o Slim: más código ajeno que aprender |
| Base de datos | MariaDB 11.8 LTS | Disponible en casi cualquier hosting y VPS; PDO la maneja bien; JSON como `LONGTEXT` con `CHECK` basta, porque no se consulta dentro del JSON | PostgreSQL (tabla siguiente) |
| Servidor | Nginx + PHP-FPM | Estándar; límite de peticiones nativo; un solo punto de entrada | Apache (más sencillo en compartido) |
| Correo | Resend en plan gratis; Pro cuando haga falta | API y SMTP sencillos; plan gratis verificado | Brevo, Amazon SES, SMTP del hosting (capítulo 8) |
| Formulario | Cloudflare Turnstile + campo trampa + límite por IP | Comprobación en el servidor; gratis según Cloudflare (verificar) | reCAPTCHA (más datos personales) |
| Copias | Diarias, cifradas, fuera del VPS, en la UE | Regla 3-2-1 (capítulo 9) | Otro proveedor de copias |

**MariaDB o PostgreSQL.** Supabase usa PostgreSQL, así que la migración tiene que traducir tipos (`uuid`, `jsonb`), pero el volumen es pequeño. Recomendación: **MariaDB**, por tres razones: hay más hostings y VPS que la ofrecen por defecto, el diseño ya estaba probado con ella el 17-10-2026 y el equipo (una persona) la conoce. Con PostgreSQL el resultado sería igual de válido; sería la elección si más adelante hace falta consultar dentro del JSON o escalar lecturas.

| Criterio | MariaDB 11.8 LTS | PostgreSQL |
|---|---|---|
| Disponibilidad en hostings baratos | Muy alta | Media |
| Conexión desde PHP (PDO) | Estándar | Estándar |
| JSON | `LONGTEXT` con `CHECK (JSON_VALID)`; sin índices sobre el contenido | `jsonb` con índices |
| Transacciones y bloqueos | InnoDB, `SELECT … FOR UPDATE` | Igual |
| Migración desde Supabase | Hay que traducir `uuid` y `jsonb` | Casi directa |
| Recomendación para Paglot | **Sí** | Alternativa válida |

## 3.3 El modelo de cuenta: del número a un token de dispositivo

Hoy el número viaja en **cada** petición y es la única credencial. En la propuesta el número sirve **solo para entrar en otro dispositivo**. Cada dispositivo recibe un token propio, revocable y caducable.

1. **Alta.** `POST /api/v1/cuentas` → `201 {numero, token}`. El servidor genera el número y lo devuelve una vez; **no lo guarda en claro**. El móvil guarda el número (para mostrarlo y copiarlo, como ahora) y el token.
2. **Uso normal.** Cada petición lleva `Authorization: Bearer <token>`. El número no vuelve a viajar.
3. **Otro dispositivo.** `POST /api/v1/cuentas/entrar` con el número → token nuevo para ese dispositivo. Límite de 20 intentos por hora y por IP.
4. **Correo (opcional).** `POST /api/v1/cuenta/correo` manda un código; `…/confirmar` lo verifica y une el correo. Un correo solo puede estar en una cuenta (409 si ya lo está). Con correo, la cuenta no caduca.
5. **Recuperación.** `POST /api/v1/recuperar` responde siempre 202. Con el código, `…/recuperar/confirmar` da un token nuevo y **cierra las sesiones anteriores**.
6. **Borrado.** `DELETE /api/v1/cuenta` borra la cuenta y, en cascada, su progreso, sus tokens y su historial.

**Qué se gana.** El número deja de estar en cada petición (y en los registros de los proxys). Cada dispositivo se puede cerrar por separado. Los intentos de entrar se limitan. La recuperación ya no entrega el número: entrega un token.

**Qué no se gana.** Quien sepa el número puede entrar. Por eso el número sigue siendo un secreto que la app debe tratar como contraseña (no mostrarlo en pantalla salvo que se toque, como ahora, y no enviarlo por canales abiertos).

**Cambio en la app.** `js/nube.js` deja de usar `rpc()` y `auth()` de Supabase y pasa a usar la API (ejemplo en el apartado 6.8). Hay que guardar el token junto al número. Las cuentas que ya existen conservan su número: el servidor lo convierte en HMAC durante la migración (capítulo 7).

## 3.4 Endpoints (equivalentes a las funciones de hoy)

Prefijo: `https://api.paglot.app/api/v1`. «Sesión» indica si hace falta el token.

| Ruta | Método | Sesión | Cuerpo | Respuesta de éxito | Sustituye a |
|---|---|---|---|---|---|
| `/salud` | GET | No | — | `200 {ok, hora}` | — |
| `/cuentas` | POST | No | `{}` | `201 {numero, token}` | `paglot_guardar` (cuenta nueva) |
| `/cuentas/entrar` | POST | No | `{numero}` | `200 {token}` | `paglot_existe` + `paglot_leer` |
| `/cuenta` | GET | Sí | — | `200 {correo, verificado, creada}` | `paglot_correo` (enmascarado) |
| `/cuenta` | DELETE | Sí | — | `204` | `paglot_borrar` |
| `/cuenta/correo` | POST | Sí | `{correo}` | `202 {ok}` | `auth/otp` |
| `/cuenta/correo/confirmar` | POST | Sí | `{correo, codigo}` | `200 {correo}` (enmascarado) | `paglot_vincular` |
| `/recuperar` | POST | No | `{correo}` | `202 {ok}` (siempre) | `auth/otp` |
| `/recuperar/confirmar` | POST | No | `{correo, codigo}` | `200 {token}` | `paglot_mi_numero` |
| `/progreso` | GET | Sí | — | `200 {cursos: {es: {datos, version, actualizado}, …}}` | `paglot_leer` (todos los cursos) |
| `/progreso/{curso}` | GET | Sí | — | `200 {datos, version, actualizado}` | `paglot_leer` |
| `/progreso/{curso}` | PUT | Sí | `{datos, version}` | `200 {version}` | `paglot_guardar` (con versión) |
| `/progreso/{curso}` | DELETE | Sí | — | `204` | (nuevo) |
| `/errores` | POST | No | `{mensaje, donde, pantalla, curso, version, navegador}` | `204` | `paglot_error` |
| `/ideas` | GET | Opcional | — | `200 {ideas: [...]}` (con `votada` si hay sesión) | `paglot_ideas_ver` |
| `/ideas` | POST | Sí | `{texto, autora?}` | `201 {id}` | `paglot_idea_nueva` |
| `/ideas/{id}/voto` | PUT | Sí | — | `204` | `paglot_votar` (sí) |
| `/ideas/{id}/voto` | DELETE | Sí | — | `204` | `paglot_votar` (no) |
| `/ideas/{id}/denuncia` | POST | Sí | — | `204` | `paglot_denunciar` |
| `/contacto` | POST | No | `{correo, asunto, mensaje, usuario?, turnstile, web}` | `202 {ok}` | (nuevo; hoy no envía nada) |

**Respuestas de error.** Siempre `{"error": "codigo", …}`. Los códigos reutilizan los del diseño de 17-10-2026, para que la app los traduzca con `t('clave')`.

| Estado | Código | Cuándo |
|---|---|---|
| 401 | `sin_sesion` | Falta el token, no existe o ha caducado |
| 403 | `origen` | El navegador llama desde un origen que no está en la lista |
| 404 | `no_existe` / `no_encontrado` | Número desconocido (`entrar`) o idea inexistente |
| 405 | `metodo` | La ruta existe pero no con ese método |
| 409 | `conflicto` | La versión enviada no es la de la nube (con `datos` de la nube) |
| 409 | `retroceso` | La copia nueva pierde más de la mitad de lo guardado (salvo reinicio) |
| 409 | `correo_en_uso` | El correo ya está en otra cuenta |
| 400 | `codigo_invalido` | Código incorrecto, caducado o gastado |
| 413 | `demasiado_grande` | Cuerpo por encima del máximo (16 KB, o 2 MB en progreso) |
| 415 | `tipo_contenido` | No es `application/json` |
| 422 | `datos_invalidos` (con `campo`, `motivo`) | Falta un campo o no encaja |
| 422 | `enlace`, `comprobacion_fallida` | Enlace en una idea; Turnstile no válido |
| 429 | `demasiados` (con `Retry-After`) | Límite superado |

| Estado | Código | Cuándo |
|---|---|---|
| 500 | `interno` | Error no previsto (sin detalles) |
| 503 | — | Base de datos no disponible (la salud la devuelve así en la versión final) |

### 3.5 CORS y orígenes permitidos

| Origen | Quién lo usa | Notas |
|---|---|---|
| `https://paglot.app` | Web | Dominio definitivo |
| `https://localhost` | APK de Android | Capacitor usa este origen por defecto en Android (`capacitor.config.json` no lo cambia). Comprobarlo con la versión de Capacitor instalada |
| `capacitor://localhost` | App de iOS | Por defecto en iOS |
| `http://localhost:8100` | Pruebas en el ordenador | Quitarlo en producción |

Sin cabecera `Origin` (curl, el propio servidor) no hay CORS que aplicar. Nunca `*`, nunca reflejar cualquier origen.

### 3.6 Límites por defecto

Están en `config/config.php` (apartado 5). Resumen: crear cuenta 10 por hora y IP; entrar con número 20 por hora y IP; código por IP 20 por hora y por correo 5 por hora; intentos de código 5 por código; contacto 5 por hora y IP; errores 100 por hora y IP; ideas 3 al día por cuenta; votos 200 por hora por cuenta; denuncias 20 por hora por cuenta. Cuerpo máximo 16 KB, y 2 MB para el progreso de un curso.

---

## 4. Esquema SQL completo (MariaDB)

El esquema está en `sql/esquema.sql`. Se ejecuta varias veces sin borrar nada.

`sql/esquema.sql`

```
-- ======================================================================
-- Paglot · base de datos de la API propia (MariaDB 10.11+ o MySQL 8.0+). Propuesta de 19-10-2026.
-- Se puede ejecutar varias veces: no borra datos. Horas siempre en UTC.
-- El número de cuenta NO se guarda: solo su HMAC con el secreto del servidor (ver Seguridad::hmac()).
-- ======================================================================

-- Una fila por cuenta anónima o verificada.
CREATE TABLE IF NOT EXISTS cuentas (
  id                CHAR(36)     CHARACTER SET ascii NOT NULL,   -- UUID v4 interno (nunca se muestra)
  numero_hmac       CHAR(64)     CHARACTER SET ascii NOT NULL,   -- HMAC-SHA256(número, secreto)
  correo_hash       CHAR(64)     CHARACTER SET ascii NULL,       -- HMAC-SHA256(correo, secreto); solo si está verificado
  correo_mascara    VARCHAR(40)  NULL,                           -- «j***@gmail.com», lo único del correo en claro
  correo_verificado DATETIME(3)  NULL,                           -- con correo verificado la cuenta no caduca
  creada            DATETIME(3)  NOT NULL,
  usada             DATETIME(3)  NOT NULL,                       -- para borrar las anónimas sin uso a los 7 días
  PRIMARY KEY (id),
  UNIQUE KEY numero (numero_hmac),
  UNIQUE KEY correo (correo_hash),
  KEY usada (usada)
) ENGINE=InnoDB DEFAULT CHARSET=ascii COLLATE=ascii_bin;

-- Un token por dispositivo (180 días). Del token solo se guarda su huella SHA-256.
CREATE TABLE IF NOT EXISTS tokens_dispositivo (
  token_hash BINARY(32)  NOT NULL,
  cuenta_id  CHAR(36)    CHARACTER SET ascii NOT NULL,
  creado     DATETIME(3) NOT NULL,
  usado      DATETIME(3) NOT NULL,
  caduca     DATETIME(3) NOT NULL,
  PRIMARY KEY (token_hash),
  KEY cuenta (cuenta_id),
  KEY caduca (caduca),
  CONSTRAINT tokens_cuenta FOREIGN KEY (cuenta_id) REFERENCES cuentas (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=ascii COLLATE=ascii_bin;

-- El progreso: una fila por cuenta y curso. «datos» es el JSON de js/progress.js (MariaDB lo guarda como LONGTEXT).
CREATE TABLE IF NOT EXISTS progreso (
  cuenta_id   CHAR(36)     CHARACTER SET ascii NOT NULL,
  curso       VARCHAR(8)   CHARACTER SET ascii NOT NULL,
  datos       LONGTEXT     CHARACTER SET utf8mb4 NOT NULL CHECK (JSON_VALID(datos)),
  version     INT UNSIGNED NOT NULL,
  actualizado DATETIME(3)  NOT NULL,
  PRIMARY KEY (cuenta_id, curso),
  CONSTRAINT progreso_cuenta FOREIGN KEY (cuenta_id) REFERENCES cuentas (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Copias del progreso anterior (como mucho una por hora y curso; se borran a los 14 días).
CREATE TABLE IF NOT EXISTS progreso_historial (
```

```
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  cuenta_id   CHAR(36)     CHARACTER SET ascii NOT NULL,
  curso       VARCHAR(8)   CHARACTER SET ascii NOT NULL,
  datos       LONGTEXT     CHARACTER SET utf8mb4 NOT NULL,
  version     INT UNSIGNED NOT NULL,
  guardado    DATETIME(3)  NOT NULL,
  PRIMARY KEY (id),
  KEY cuenta_curso (cuenta_id, curso, guardado),
  KEY guardado (guardado),
  CONSTRAINT historial_cuenta FOREIGN KEY (cuenta_id) REFERENCES cuentas (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Códigos de 6 cifras por correo: uno vigente por correo y acción. Se guarda su HMAC, nunca el código.
CREATE TABLE IF NOT EXISTS codigos (
  correo_hash CHAR(64)  CHARACTER SET ascii NOT NULL,
  accion      ENUM('vincular', 'recuperar') NOT NULL,
  huella      BINARY(32) NOT NULL,
  intentos    TINYINT UNSIGNED NOT NULL DEFAULT 0,
  caduca      DATETIME(3) NOT NULL,
  PRIMARY KEY (correo_hash, accion),
  KEY caduca (caduca)
) ENGINE=InnoDB DEFAULT CHARSET=ascii COLLATE=ascii_bin;

-- Contadores de intentos por ventana fija. La clave es «tipo:HMAC(valor)»: ni IP ni correo a la vista.
CREATE TABLE IF NOT EXISTS limites (
  clave   VARCHAR(100) CHARACTER SET ascii NOT NULL,
  inicio  DATETIME(3)  NOT NULL,
  cuenta  INT UNSIGNED NOT NULL DEFAULT 0,
  PRIMARY KEY (clave),
  KEY inicio (inicio)
) ENGINE=InnoDB DEFAULT CHARSET=ascii COLLATE=ascii_bin;

-- Errores de la app, anónimos. Los iguales del mismo día son una fila con «veces».
CREATE TABLE IF NOT EXISTS errores (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  huella      BINARY(32)   NOT NULL,                -- día + mensaje + dónde + versión + navegador
  dia         DATE         NOT NULL,
  mensaje     VARCHAR(300) NOT NULL,
  donde       VARCHAR(200) NOT NULL DEFAULT '',
  pantalla    VARCHAR(80)  NOT NULL DEFAULT '',
  curso       VARCHAR(8)   NOT NULL DEFAULT '',
  version     VARCHAR(40)  NOT NULL DEFAULT '',
  navegador   VARCHAR(60)  NOT NULL DEFAULT '',
  veces       INT UNSIGNED NOT NULL DEFAULT 1,
  primera     DATETIME(3)  NOT NULL,
  ultima      DATETIME(3)  NOT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY huella (huella),
  KEY ultima (ultima)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Laboratorio de ideas. «cuenta_id» se pone a NULL si la cuenta se borra (la idea sigue).
CREATE TABLE IF NOT EXISTS ideas (
  id          CHAR(36)     CHARACTER SET ascii NOT NULL,
  texto       VARCHAR(180) NOT NULL,
  autora      VARCHAR(32)  NOT NULL DEFAULT '',
  cuenta_id   CHAR(36)     CHARACTER SET ascii NULL,
  oculta      TINYINT(1)   NOT NULL DEFAULT 0,
  creada      DATETIME(3)  NOT NULL,
  PRIMARY KEY (id),
  KEY visibles (oculta, creada),
  CONSTRAINT ideas_cuenta FOREIGN KEY (cuenta_id) REFERENCES cuentas (id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Un voto por cuenta y idea.
CREATE TABLE IF NOT EXISTS ideas_votos (
  idea_id    CHAR(36) CHARACTER SET ascii NOT NULL,
  cuenta_id  CHAR(36) CHARACTER SET ascii NOT NULL,
  PRIMARY KEY (idea_id, cuenta_id),
  KEY cuenta (cuenta_id),
  CONSTRAINT votos_idea   FOREIGN KEY (idea_id)   REFERENCES ideas (id)   ON DELETE CASCADE,
  CONSTRAINT votos_cuenta FOREIGN KEY (cuenta_id) REFERENCES cuentas (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=ascii COLLATE=ascii_bin;

-- Denuncias: una por cuenta y idea. Con tres, la idea se oculta.
CREATE TABLE IF NOT EXISTS ideas_denuncias (
```

```
  idea_id    CHAR(36) CHARACTER SET ascii NOT NULL,
  cuenta_id  CHAR(36) CHARACTER SET ascii NOT NULL,
  PRIMARY KEY (idea_id, cuenta_id),
  KEY cuenta (cuenta_id),
  CONSTRAINT denuncias_idea   FOREIGN KEY (idea_id)   REFERENCES ideas (id)   ON DELETE CASCADE,
  CONSTRAINT denuncias_cuenta FOREIGN KEY (cuenta_id) REFERENCES cuentas (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=ascii COLLATE=ascii_bin;
```

## 4.1 Qué guarda cada tabla y cuánto dura

| Tabla | Qué guarda | Retención | Quién la limpia |
|---|---|---|---|
| `cuentas` | Número (HMAC), correo (HMAC y enmascarado), fechas | Anónimas: 7 días sin uso. Con correo: sin límite | `tareas/limpieza.php` |
| `tokens_dispositivo` | Huella SHA-256 del token, cuenta, fechas | 180 días desde la creación, o hasta cerrar sesión | `limpieza.php` |
| `progreso` | JSON de cada curso, versión, fecha | Mientras exista la cuenta | Al borrar la cuenta (cascada) |
| `progreso_historial` | Copia del progreso anterior, como mucho una por hora | 14 días | `limpieza.php` |
| `codigos` | HMAC de cada código, intentos, caducidad | 10 minutos | `limpieza.php` |
| `limites` | Contadores por ventana, clave con HMAC | 2 días | `limpieza.php` |
| `errores` | Errores anónimos, con contador por día | 90 días | `limpieza.php` |
| `ideas`, `ideas_votos`, `ideas_denuncias` | Propuestas, votos y denuncias | Mientras existan (las ocultas, a revisión) | Manual |

Nota sobre datos personales: en `cuentas` no hay número en claro, no hay correo en claro y no hay IP. En `errores` no hay cuenta ni correo; el texto se limpia en la API (apartado 5, `Errores`). En `ideas` la autora es texto libre y aparece en público: hay que decirlo en el aviso y dejar la opción de no poner nombre.

## 4.2 Por qué el número se guarda como HMAC y no en claro

Un número tiene $9 \times 10^{15}$ valores posibles. Si se guardara su SHA-256 **sin clave**, un atacante que copiara la base de datos podría calcular el SHA-256 de todos los números. Con una sola GPU de gama alta (que calcula miles de millones de SHA-256 por segundo; estimación), recorrer todo el espacio llevaría unos **10 días**. Eso es mala idea.

Con **HMAC-SHA256** y un secreto que solo está en el servidor (`PAGLOT_SECRETO`, fuera de la carpeta pública), copiar la base no basta: sin el secreto no hay forma de probar nada. La búsqueda sigue funcionando porque el HMAC es determinista: el servidor calcula el HMAC del número que llega y busca por índice (`numero_hmac`). El mismo principio se usa para el correo (`correo_hash`) y para las claves de los contadores (`limites.clave`).

**Consecuencias y cuidados.**

- El secreto **no puede cambiar sin recalcular** todas las huellas. Para poder rotarlo, la tabla `cuentas` tendría una columna de versión de clave; hoy no está prevista (queda como mejora).
- El secreto se guarda en una variable de entorno o en `config/config.php` con permisos 600, **nunca en Git**. Si se pierde, todas las cuentas quedan inaccesibles: hay que guardarlo también en un gestor de contraseñas.
- Con el correo solo en forma de HMAC y enmascarado, la API **no puede escribir al correo** salvo que lo vuelva a escribir la persona. Es una decisión a favor de la privacidad: no hay avisos de seguridad automáticos por correo. Si se quiere, se añade una columna con el correo cifrado (AES-256-GCM) con otra clave.

## 4.3 Índices

| Tabla | Índice | Para qué |
|---|---|---|
| `cuentas` | `numero` (único, sobre HMAC) | Entrar con el número |
| `cuentas` | `correo` (único, sobre HMAC) | Recuperar, y que un correo esté solo en una cuenta |
| `cuentas` | `usada` | Borrar las anónimas caducadas sin recorrer la tabla entera |
| `tokens_dispositivo` | clave primaria sobre la huella | Autenticar cada petición |
| `tokens_dispositivo` | `caduca` | Limpieza |
| `progreso` | `(cuenta_id, curso)` | Leer y guardar un curso |
| `progreso_historial` | `(cuenta_id, curso, guardado)` | Comprobar la copia de la última hora |
| `errores` | `huella` (único) | Sumar en lugar de duplicar |
| `ideas` | `(oculta, creada)` | Listado público |

## 4.4 Permisos de la base de datos

Dos usuarios, como en el diseño de 17-10-2026: `paglot_app` para la API y `paglot_tareas` para el cron. Ninguno puede crear ni borrar tablas. El script está en `sql/permisos.sql`.

`sql/permisos.sql`

```
-- ====================================================================
-- Usuarios de la base y permisos mínimos. Cambiar las dos contraseñas por valores largos y aleatorios
-- (las mismas que pongas en config/config.php o en las variables PAGLOT_BD_CONTRASENA*).
-- La API no puede crear ni borrar tablas ni tocar otras bases. Las tareas de cron usan el segundo usuario.
-- En el ordenador (Windows) la conexión llega por 127.0.0.1: por eso se crean los dos equipos.
-- ====================================================================
CREATE USER IF NOT EXISTS 'paglot_app'@'localhost'     IDENTIFIED BY 'CAMBIAR_POR_UNA_LARGA_Y_ALEATORIA';
CREATE USER IF NOT EXISTS 'paglot_app'@'127.0.0.1'     IDENTIFIED BY 'CAMBIAR_POR_UNA_LARGA_Y_ALEATORIA';
CREATE USER IF NOT EXISTS 'paglot_tareas'@'localhost'  IDENTIFIED BY 'CAMBIAR_POR_OTRA_LARGA_Y_ALEATORIA';
CREATE USER IF NOT EXISTS 'paglot_tareas'@'127.0.0.1'  IDENTIFIED BY 'CAMBIAR_POR_OTRA_LARGA_Y_ALEATORIA';

GRANT SELECT, INSERT, UPDATE, DELETE ON paglot.* TO 'paglot_app'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE ON paglot.* TO 'paglot_app'@'127.0.0.1';
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON paglot.* TO 'paglot_tareas'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE ON paglot.* TO 'paglot_tareas'@'127.0.0.1';
FLUSH PRIVILEGES;
```

En Windows, la conexión llega por `127.0.0.1`, que MariaDB trata como otro equipo distinto de `localhost`; por eso se crean los dos.

## 5. Código PHP de ejemplo

**Advertencia.** Este código no se ha ejecutado: en el equipo no hay PHP. Está escrito con cuidado y con las decisiones del diseño, pero antes de usarlo hay que pasar las pruebas del capítulo 6 y revisarlo con alguien que sepa PHP. Hay un punto que ya se ha corregido en la revisión: los contadores de límites y de códigos se guardan **fuera** de la transacción que lanza el error, porque si no MariaDB los deshace y el límite nunca se cumpliría.

### 5.1 Estructura de carpetas

```
api/                          (fuera de la web pública: solo Nginx sirve api/public)
  config/config.php           copia de config.ejemplo.php, permisos 600, no va a Git
  public/index.php            la única puerta de entrada
  src/
    Arranque.php              configuración, autocargador, errores sin detalles
    Http.php                  ErrorApi, Respuesta, Solicitud (JSON, tamaños, token, IP)
    Db.php                    PDO con consultas preparadas y transacciones
    Seguridad.php             tokens, HMAC, números, correos, códigos
    Limites.php               contadores por ventana (IP, correo, cuenta)
    Rutas.php                 tabla de rutas y CORS
    Cuentas.php               alta, entrar con número, ver, borrar, token
    Progreso.php              progreso por curso con versión, 409 y retroceso
    Correo.php                códigos de correo, recuperación, envío (PHPMailer)
    Errores.php               informes de errores anónimos
    Ideas.php                 laboratorio de ideas: listar, proponer, votar, denunciar
    Contacto.php              formulario: Turnstile, campo trampa, correo al equipo
  tareas/limpieza.php         cron nocturno
  sql/esquema.sql, sql/permisos.sql
  buzon/                      solo en pruebas: los correos se guardan como archivos
  vendor/                     Composer (phpmailer/phpmailer)
```

Instalación de la dependencia, dentro de `api/`: `composer require phpmailer/phpmailer` (la versión 6 de PHPMailer; no se fija aquí la versión exacta).

### 5.2 Decisiones de diseño que el código cumple

1. **Consultas preparadas siempre** (`Db`). Ningún dato de la petición se pega en el SQL.
2. **Nada de detalles hacia fuera.** Los errores inesperados devuelven `{"error":"interno"}` con código 500. En el registro del servidor solo quedan el tipo de error y el archivo; nunca el cuerpo de la petición ni los datos.
3. **Tamaños antes que nada.** Se mira `Content-Length` y se lee como mucho el máximo. El progreso admite 2 MB; el resto, 16 KB.
4. **Solo** `application/json`**.** Así el navegador siempre pregunta antes por CORS.
5. **Lista cerrada de orígenes**, sin `*`.
6. **Token de 256 bits** en la cabecera, nunca en la URL. En la base solo va su SHA-256.
7. **Números y códigos aleatorios con** `random_bytes` **/** `random_int`**.**
8. **Correos después de responder** (`fastcgi_finish_request`). La respuesta tarda igual haya correo o no, y así no se puede saber si un correo tiene cuenta por el tiempo.
9. **Respuesta igual para correos con y sin cuenta** en la recuperación (202 siempre).
10. **Códigos de correo**: se guarda solo su HMAC, valen 10 minutos, solo vale el último pedido, se gastan al acertar y se borran tras 5 fallos. El contador de fallos se graba antes de responder con error.
11. **Límites** por IP y por cuenta, con la espera en `Retry-After`.
12. **Progreso con versión.** Si la versión no coincide, 409 con la copia de la nube para que la app la junte. Si la copia nueva tiene menos de la mitad de lecciones y palabras (y no es un reinicio), 409 `retroceso`.
13. **Copia de lo anterior** como mucho una vez por hora, durante 14 días.
14. **Errores sin datos personales**: se quitan correos y números largos antes de guardar.
15. **Contacto sin base de datos**: el mensaje va por correo al equipo, con la respuesta dirigida a la persona, y no se guarda en ningún sitio.

### 5.3 Lo que falta en el ejemplo (para la versión definitiva)

- **Exportar los datos de la cuenta** (`GET /cuenta/exportar`), necesario para el derecho de portabilidad del artículo 20 del RGPD.
- **Cola de reintentos** de correos (la tabla `correos_pendientes` del diseño de 17-10-2026 sirve), porque hoy un fallo de SMTP solo se registra.
- **Rotación del secreto** (apartado 4.2).
- **Pruebas automáticas** (apartado 6.7).
- **Comprobación de Turnstile en la creación de cuentas**, si el abuso lo exige.
- **Registro de eventos sin datos personales** (la tabla `eventos` del diseño de 17-10-2026).

### 5.4 Código

`config/config.php (copia fuera de public/)`

```
<?php
// Configuración de la API. Se copia como config/config.php FUERA de public/, con permisos 600 y el usuario de PHP
// como dueño. No va a Git. Los secretos pueden venir de variables de entorno (PAGLOT_*).
```

```
declare(strict_types=1);

$env = static fn (string $nombre, string $defecto = ''): string => getenv($nombre) !== false ? (string) getenv($nombre) : $defecto;

return [
    'entorno' => $env('PAGLOT_ENTORNO', 'produccion'),        // «produccion» no muestra detalles de los errores
    'secreto' => $env('PAGLOT_SECRETO'),                       // 64 o más caracteres aleatorios: HMAC de números, correos y IP

    'bd' => [
        'dsn'               => 'mysql:host=127.0.0.1;port=3306;dbname=paglot;charset=utf8mb4',
        'usuario'           => $env('PAGLOT_BD_USUARIO', 'paglot_app'),
        'contrasena'        => $env('PAGLOT_BD_CONTRASENA'),
        'tareas_usuario'    => 'paglot_tareas',                // solo para las tareas de cron
        'tareas_contrasena' => $env('PAGLOT_BD_CONTRASENA_TAREAS'),
    ],

    // CORS con lista cerrada. Nunca «*». El APK de Android usa https://localhost; el de iOS, capacitor://localhost.
    'origenes' => [
        'https://paglot.app',
        'https://localhost',
        'capacitor://localhost',
        'http://localhost:8100',                               // desarrollo en el ordenador
    ],

    // Proxys de confianza. Vacío con Nginx + PHP-FPM (REMOTE_ADDR ya es la IP del cliente). Con Cloudflare delante,
    // poner sus rangos y leer CF-Connecting-IP (cabecera_ip = 'HTTP_CF_CONNECTING_IP').
    'proxys'      => [],
    'cabecera_ip' => 'HTTP_X_FORWARDED_FOR',

    'correo' => [
        'modo'     => $env('PAGLOT_CORREO_MODO', 'smtp'),      // «archivo» guarda los correos en buzon/ (pruebas)
        'desde'    => 'hola@paglot.app',
        'equipo'   => 'soporte@paglot.app',                    // adónde llegan los mensajes del formulario de contacto
        'buzon'    => dirname(__DIR__) . '/buzon',
        'smtp' => [
            'host'       => $env('PAGLOT_SMTP_HOST'),
            'puerto'     => 587,                               // STARTTLS
            'usuario'    => $env('PAGLOT_SMTP_USUARIO'),
            'contrasena' => $env('PAGLOT_SMTP_CONTRASENA'),
        ],
    ],

    'turnstile' => ['secreto' => $env('PAGLOT_TURNSTILE_SECRETO')],   // vacío = no se comprueba (solo pruebas)

    'cuerpo_max'   => 16384,                                   // bytes de cualquier petición que no sea progreso
    'progreso_max' => 2097152,                                 // 2 MB por curso

    // Límites: tipo => [máximo de intentos, ventana en segundos]
    'limites' => [
        'crear_cuenta'    => [10, 3600],
        'entrar_numero'   => [20, 3600],
        'codigo_ip'       => [20, 3600],
        'codigo_correo'   => [5, 3600],
        'recuperar_ip'    => [10, 3600],
        'contacto'        => [5, 3600],
        'errores'         => [100, 3600],
        'idea'            => [3, 86400],
        'voto'            => [200, 3600],
        'denuncia'        => [20, 3600],
    ],
];
```

public/index.php

```
<?php
// La única puerta de entrada: Nginx solo sirve esta carpeta. El código, la configuración y los registros quedan fuera.
declare(strict_types=1);

require dirname(__DIR__) . '/src/Arranque.php';

$respuesta = \Paglot\Rutas::despachar(\Paglot\Solicitud::actual());
$respuesta->enviar();

// Los correos se envían después de responder: así la respuesta tarda lo mismo haya correo o no.
if (function_exists('fastcgi_finish_request')) fastcgi_finish_request();
\Paglot\Correo::enviarEncolados();
```

src/Arranque.php

```
<?php
// Arranque común: configuración, autocargador (Composer para PHPMailer y, si no, el propio) y errores sin detalles.
declare(strict_types=1);

namespace Paglot;

const RAIZ = __DIR__ . '/..';

if (is_file(RAIZ . '/vendor/autoload.php')) require RAIZ . '/vendor/autoload.php';   // PHPMailer (composer require phpmailer/phpmailer)

require __DIR__ . '/Http.php';   // ErrorApi, Respuesta y Solicitud van en el mismo archivo

spl_autoload_register(static function (string $clase): void {
    if (!str_starts_with($clase, 'Paglot\\')) return;
    $archivo = __DIR__ . '/' . str_replace('\\', '/', substr($clase, strlen('Paglot\\'))) . '.php';
    if (is_file($archivo)) require $archivo;
});

/** Configuración con rutas tipo «bd.dsn» o «limites.codigo_ip». */
final class Config
{
    private static array $valores = [];

    public static function iniciar(array $valores): void
    {
        self::$valores = $valores;
    }

    public static function get(string $ruta): mixed
    {
        $actual = self::$valores;
        foreach (explode('.', $ruta) as $clave) {
            if (!is_array($actual) || !array_key_exists($clave, $actual)) return null;
            $actual = $actual[$clave];
        }
        return $actual;
    }
}

date_default_timezone_set('UTC');   // las horas de la base van en UTC
ini_set('display_errors', '0');     // nunca se muestran errores al exterior
error_reporting(E_ALL);

set_exception_handler(static function (\Throwable $e): void {
    // Se registra el tipo y el sitio, nunca el cuerpo de la petición ni los datos.
    error_log('[paglot] ' . $e::class . ' en ' . $e->getFile() . ':' . $e->getLine());
    http_response_code(500);
    header('Content-Type: application/json; charset=utf-8');
    echo '{"error":"interno"}';
});

Config::iniciar(require RAIZ . '/config/config.php');   // fuera de public/
```

src/Http.php

```
<?php
// Error que se devuelve a la app: un código corto y, si hace falta, unos datos. Nunca detalles internos.
declare(strict_types=1);

namespace Paglot;

final class ErrorApi extends \RuntimeException
{
    public function __construct(
        public readonly int $estado,
        public readonly string $codigo,
        public readonly array $extra = [],
        public readonly array $cabeceras = [],
    ) {
        parent::__construct($codigo);
    }
}

/** La respuesta: estado, cuerpo JSON y cabeceras. */
final class Respuesta
```

```
{
    public function __construct(public int $estado, public array $cuerpo = [], public array $cabeceras = []) {}

    public function conCabeceras(array $extra): self
    {
        $this->cabeceras += $extra;
        return $this;
    }

    public function enviar(): void
    {
        http_response_code($this->estado);
        header('Cache-Control: no-store');
        header('X-Content-Type-Options: nosniff');
        header('Referrer-Policy: no-referrer');
        foreach ($this->cabeceras as $nombre => $valor) header("$nombre: $valor");
        if ($this->estado === 204) return;
        header('Content-Type: application/json; charset=utf-8');
        echo json_encode($this->cuerpo, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES | JSON_THROW_ON_ERROR);
    }
}

/** La petición que llega: método, ruta, IP real, origen y el cuerpo JSON (ya comprobado). */
final class Solicitud
{
    private ?\stdClass $leido = null;

    private function __construct(
        public readonly string $metodo,
        public readonly string $ruta,
        public readonly string $ip,
        public readonly ?string $origen,
        private readonly array $servidor,
    ) {}

    public static function actual(): self
    {
        $s = $_SERVER;
        $ruta = rawurldecode((string) (parse_url((string) ($s['REQUEST_URI'] ?? '/'), PHP_URL_PATH) ?: '/'));
        $ip = (string) ($s['REMOTE_ADDR'] ?? '');
        $cabecera = (string) ($s[(string) Config::get('cabecera_ip')] ?? '');
        if (in_array($ip, (array) Config::get('proxys'), true) && $cabecera !== '') {
            $partes = explode(',', $cabecera);
            $ip = trim((string) end($partes));   // la última la ha añadido nuestro proxy
        }
        $origen = $s['HTTP_ORIGIN'] ?? null;
        return new self(
            strtoupper((string) ($s['REQUEST_METHOD'] ?? 'GET')),
            $ruta,
            filter_var($ip, FILTER_VALIDATE_IP) ? $ip : '0.0.0.0',
            is_string($origen) ? $origen : null,
            $s,
        );
    }

    /** Token de dispositivo: «Authorization: Bearer» con 43 caracteres. Nunca viaja en la URL. */
    public function token(): ?string
    {
        $cab = (string) ($this->servidor['HTTP_AUTHORIZATION'] ?? $this->servidor['REDIRECT_HTTP_AUTHORIZATION'] ?? '');
        return preg_match('/^Bearer ([A-Za-z0-9_-]{43})$/', $cab, $m) === 1 ? $m[1] : null;
    }

    /** El cuerpo como objeto JSON. Exige application/json y no pasa del tamaño máximo. */
    public function cuerpo(int $maximo): \stdClass
    {
        if ($this->leido !== null) return $this->leido;
        if ((int) ($this->servidor['CONTENT_LENGTH'] ?? 0) > $maximo) throw new ErrorApi(413, 'demasiado_grande');
        $crudo = (string) file_get_contents('php://input', false, null, 0, $maximo + 1);
        if (strlen($crudo) > $maximo) throw new ErrorApi(413, 'demasiado_grande');
        $tipo = strtolower(trim(explode(';', (string) ($this->servidor['CONTENT_TYPE'] ?? ''))[0]));
        if ($tipo !== 'application/json') throw new ErrorApi(415, 'tipo_contenido');
        try {
            $datos = json_decode($crudo === '' ? '{}' : $crudo, false, 64, JSON_THROW_ON_ERROR);
        } catch (\JsonException) {
            throw new ErrorApi(400, 'json_invalido');
        }
```

```
        if (!$datos instanceof \stdClass) throw new ErrorApi(400, 'json_invalido');
        return $this->leido = $datos;
    }

    /** Un campo de texto. Sin el campo y obligatorio, error; si no es texto o pasa del máximo, error. */
    public static function texto(\stdClass $c, string $campo, int $maximo, bool $obligatorio = true): ?string
    {
        $valor = $c->$campo ?? null;
        if ($valor === null) {
            if ($obligatorio) throw new ErrorApi(422, 'datos_invalidos', ['campo' => $campo, 'motivo' => 'falta']);
            return null;
        }
        if (!is_string($valor) || mb_strlen($valor) > $maximo) {
            throw new ErrorApi(422, 'datos_invalidos', ['campo' => $campo, 'motivo' => 'formato']);
        }
        return $valor;
    }
}
```

src/Db.php

```
<?php
// Acceso a la base de datos: PDO con consultas preparadas de verdad. Nunca se pega texto de fuera en el SQL.
declare(strict_types=1);

namespace Paglot;

use PDO;

final class Db
{
    private static ?PDO $pdo = null;

    public static function pdo(): PDO
    {
        return self::$pdo ??= self::conectar((string) Config::get('bd.usuario'), (string) Config::get('bd.contrasena'));
    }

    /** Para las tareas de cron: se conecta con el usuario de tareas (puede borrar; la API, no). */
    public static function comoTareas(): void
    {
        self::$pdo = self::conectar((string) Config::get('bd.tareas_usuario'), (string) Config::get('bd.tareas_contrasena'));
    }

    private static function conectar(string $usuario, string $contrasena): PDO
    {
        $pdo = new PDO((string) Config::get('bd.dsn'), $usuario, $contrasena, [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_EMULATE_PREPARES   => false,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        ]);
        $pdo->exec("SET time_zone = '+00:00'");
        return $pdo;
    }

    /** Una fila, o null. */
    public static function uno(string $sql, array $parametros = []): ?array
    {
        $consulta = self::pdo()->prepare($sql);
        $consulta->execute($parametros);
        $fila = $consulta->fetch();
        return $fila === false ? null : $fila;
    }

    public static function todos(string $sql, array $parametros = []): array
    {
        $consulta = self::pdo()->prepare($sql);
        $consulta->execute($parametros);
        return $consulta->fetchAll();
    }

    /** Devuelve cuántas filas ha cambiado. */
    public static function ejecutar(string $sql, array $parametros = []): int
    {
        $consulta = self::pdo()->prepare($sql);
        $consulta->execute($parametros);
```

```
        return $consulta->rowCount();
    }

    /** Todo dentro de una transacción. Si MariaDB la cancela por interbloqueo (1213), se repite. */
    public static function transaccion(callable $accion, int $intentos = 3): mixed
    {
        $pdo = self::pdo();
        for ($i = 1; ; $i++) {
            $pdo->beginTransaction();
            try {
                $resultado = $accion();
                $pdo->commit();
                return $resultado;
            } catch (\PDOException $e) {
                if ($pdo->inTransaction()) $pdo->rollBack();
                if (($e->errorInfo[1] ?? 0) !== 1213 || $i >= $intentos) throw $e;
            } catch (\Throwable $e) {
                if ($pdo->inTransaction()) $pdo->rollBack();
                throw $e;
            }
        }
    }
}
```

src/Seguridad.php

```
<?php
// Números aleatorios, tokens y huellas. Todo con random_bytes() / random_int(), nunca rand() ni mt_rand().
declare(strict_types=1);

namespace Paglot;

final class Seguridad
{
    /** Token de dispositivo: 256 bits en base64url (43 caracteres). En la base solo va su huella SHA-256. */
    public static function token(): string
    {
        return rtrim(strtr(base64_encode(random_bytes(32)), '+/', '-_'), '=');
    }

    public static function huella(string $token): string
    {
        return hash('sha256', $token, true);   // 32 bytes: BINARY(32)
    }

    /** Número de cuenta de 16 cifras. La primera nunca es 0. */
    public static function numeroNuevo(): string
    {
        $numero = (string) random_int(1, 9);
        for ($i = 1; $i < 16; $i++) $numero .= (string) random_int(0, 9);
        return $numero;
    }

    /** Huella con el secreto del servidor. Sirve para buscar sin guardar el valor: sin el secreto no hay forma de probar nada. */
    public static function hmac(string $valor): string
    {
        return hash_hmac('sha256', $valor, (string) Config::get('secreto'));
    }

    public static function correoNormal(string $correo): ?string
    {
        $limpio = mb_strtolower(trim($correo));
        return mb_strlen($limpio) <= 254 && filter_var($limpio, FILTER_VALIDATE_EMAIL) ? $limpio : null;
    }

    /** «j***@gmail.com»: lo único del correo que se guarda en claro. */
    public static function enmascarar(string $correo): string
    {
        [$usuario, $dominio] = explode('@', $correo, 2);
        return mb_substr($usuario, 0, 1) . '***@' . $dominio;
    }

    public static function codigo(): string
    {
        return str_pad((string) random_int(0, 999999), 6, '0', STR_PAD_LEFT);
    }
```

```
    /** UUID versión 4 (aleatorio): los ids de cuenta no se pueden adivinar. */
    public static function uuid(): string
    {
        $b = random_bytes(16);
        $b[6] = chr((ord($b[6]) & 0x0f) | 0x40);
        $b[8] = chr((ord($b[8]) & 0x3f) | 0x80);
        return vsprintf('%s%s-%s-%s-%s-%s%s%s', str_split(bin2hex($b), 4));
    }
}
```

src/Limites.php

```
<?php
// Límites por ventana fija. El valor (IP, correo, cuenta) entra como HMAC: en la tabla no hay IPs ni correos.
declare(strict_types=1);

namespace Paglot;

final class Limites
{
    /** Cuenta un intento. Si ya se ha pasado del máximo de la ventana, responde 429 con Retry-After. */
    public static function golpe(string $tipo, string $valor, bool $lanzar = true): int
    {
        [$maximo, $ventana] = Config::get("limites.$tipo") ?? throw new \LogicException("Límite desconocido: $tipo");
        $clave = $tipo . ':' . Seguridad::hmac($tipo . "\0" . $valor);   // 64 caracteres hexadecimales

        $espera = Db::transaccion(static function () use ($clave, $maximo, $ventana): int {
            Db::ejecutar('INSERT IGNORE INTO limites (clave, inicio, cuenta) VALUES (?, UTC_TIMESTAMP(3), 0)', [$clave]);
            $fila = Db::uno('SELECT TIMESTAMPDIFF(SECOND, inicio, UTC_TIMESTAMP(3)) AS pasados, cuenta FROM limites WHERE clave = ? FOR UPDATE', [$clave]);
            $pasados = (int) $fila['pasados'];

            if ($pasados >= $ventana) {                      // ventana nueva
                Db::ejecutar('UPDATE limites SET inicio = UTC_TIMESTAMP(3), cuenta = 1 WHERE clave = ?', [$clave]);
                return 0;
            }
            $cuenta = (int) $fila['cuenta'] + 1;
            Db::ejecutar('UPDATE limites SET cuenta = ? WHERE clave = ?', [$cuenta, $clave]);
            return $cuenta <= $maximo ? 0 : max(1, $ventana - $pasados);
        });
        // El aviso va fuera de la transacción: dentro, MariaDB deshace el contador y el límite nunca se cumpliría.
        if ($espera > 0 && $lanzar) throw new ErrorApi(429, 'demasiados', ['reintentar' => $espera], ['Retry-After' => (string) $espera]);
        return $espera;
    }
}
```

src/Rutas.php

```
<?php
// La tabla de rutas y el CORS. Cada ruta lleva su método, su patrón y la acción que la atiende.
declare(strict_types=1);

namespace Paglot;

final class Rutas
{
    private const TABLA = [
        ['GET',    '#^/salud$#',                          [Cuentas::class, 'salud']],
        ['POST',   '#^/cuentas$#',                        [Cuentas::class, 'crear']],
        ['POST',   '#^/cuentas/entrar$#',                 [Cuentas::class, 'entrar']],
        ['GET',    '#^/cuenta$#',                         [Cuentas::class, 'ver']],
        ['DELETE', '#^/cuenta$#',                         [Cuentas::class, 'borrar']],
        ['POST',   '#^/cuenta/correo$#',                  [Correo::class, 'pedirVinculo']],
        ['POST',   '#^/cuenta/correo/confirmar$#',        [Correo::class, 'confirmarVinculo']],
        ['POST',   '#^/recuperar$#',                      [Correo::class, 'pedirRecuperacion']],
        ['POST',   '#^/recuperar/confirmar$#',            [Correo::class, 'confirmarRecuperacion']],
        ['GET',    '#^/progreso$#',                       [Progreso::class, 'todos']],
        ['GET',    '#^/progreso/([a-z]{2,8})$#',          [Progreso::class, 'leer']],
        ['PUT',    '#^/progreso/([a-z]{2,8})$#',          [Progreso::class, 'guardar']],
        ['DELETE', '#^/progreso/([a-z]{2,8})$#',          [Progreso::class, 'borrarCurso']],
        ['POST',   '#^/errores$#',                        [Errores::class, 'registrar']],
        ['GET',    '#^/ideas$#',                          [Ideas::class, 'listar']],
        ['POST',   '#^/ideas$#',                          [Ideas::class, 'nueva']],
        ['PUT',    '#^/ideas/([0-9a-f-]{36})/voto$#',     [Ideas::class, 'votar']],
        ['DELETE', '#^/ideas/([0-9a-f-]{36})/voto$#',     [Ideas::class, 'quitarVoto']],
        ['POST',   '#^/ideas/([0-9a-f-]{36})/denuncia$#', [Ideas::class, 'denunciar']],
        ['POST',   '#^/contacto$#',                       [Contacto::class, 'enviar']],
```

```
    ];

    public static function despachar(Solicitud $s): Respuesta
    {
        $cors = [];
        try {
            $cors = self::cors($s);   // 403 si el origen no está en la lista
            if ($s->metodo === 'OPTIONS') {
                return new Respuesta(204, [], $cors + [
                    'Access-Control-Allow-Methods' => 'GET, POST, PUT, DELETE, OPTIONS',
                    'Access-Control-Allow-Headers' => 'Authorization, Content-Type',
                    'Access-Control-Max-Age'       => '600',
                ]);
            }
            $ruta = (string) preg_replace('#^/api/v1#', '', $s->ruta);
            $caminoExiste = false;
            foreach (self::TABLA as [$metodo, $patron, $accion]) {
                if (preg_match($patron, $ruta, $m) !== 1) continue;
                $caminoExiste = true;
                if ($metodo !== $s->metodo) continue;
                array_shift($m);
                return $accion($s, ...$m)->conCabeceras($cors);
            }
            throw new ErrorApi($caminoExiste ? 405 : 404, $caminoExiste ? 'metodo' : 'no_encontrado');
        } catch (ErrorApi $e) {
            return new Respuesta($e->estado, ['error' => $e->codigo] + $e->extra, $e->cabeceras + $cors);
        } catch (\Throwable $e) {
            error_log('[paglot] ' . $e::class . ' en ' . $e->getFile() . ':' . $e->getLine());
            return new Respuesta(500, ['error' => 'interno'], $cors);
        }
    }

    /** Solo los orígenes de la lista. Sin cabecera Origin (curl, el servidor) no hay CORS que aplicar. */
    private static function cors(Solicitud $s): array
    {
        if ($s->origen === null) return [];
        if (!in_array($s->origen, (array) Config::get('origenes'), true)) throw new ErrorApi(403, 'origen');
        return [
            'Access-Control-Allow-Origin'   => $s->origen,
            'Access-Control-Expose-Headers' => 'Retry-After',
            'Vary'                          => 'Origin',
        ];
    }
}
```

src/Cuentas.php

```
<?php
// Cuentas: crear, entrar con el número, ver, borrar y el token de dispositivo.
declare(strict_types=1);

namespace Paglot;

final class Cuentas
{
    public static function salud(Solicitud $s): Respuesta
    {
        Db::uno('SELECT 1 AS ok');
        return new Respuesta(200, ['ok' => true, 'hora' => gmdate('c')]);
    }

    /** Crea una cuenta anónima. El servidor hace el número, lo devuelve una sola vez y no lo guarda en claro. */
    public static function crear(Solicitud $s): Respuesta
    {
        Limites::golpe('crear_cuenta', $s->ip);
        for ($intento = 1; ; $intento++) {
            $numero = Seguridad::numeroNuevo();
            $id = Seguridad::uuid();
            try {
                Db::ejecutar(
                    'INSERT INTO cuentas (id, numero_hmac, creada, usada) VALUES (?, ?, UTC_TIMESTAMP(3), UTC_TIMESTAMP(3))',
                    [$id, Seguridad::hmac($numero)],
                );
                break;
            } catch (\PDOException $e) {
                // 1062: el número ya existe (casi imposible). Se prueba otro, como mucho cinco veces.
```

```
                if (($e->errorInfo[1] ?? 0) !== 1062 || $intento >= 5) throw $e;
            }
        }
        return new Respuesta(201, ['numero' => $numero, 'token' => self::nuevoToken($id)]);
    }

    /** Entra en otro dispositivo con el número. Devuelve un token de dispositivo, que se puede revocar. */
    public static function entrar(Solicitud $s): Respuesta
    {
        Limites::golpe('entrar_numero', $s->ip);   // cuenta también los intentos fallidos
        $numero = (string) preg_replace('/\D/', '', Solicitud::texto($s->cuerpo(256), 'numero', 32));
        if (preg_match('/^[1-9][0-9]{15}$/', $numero) !== 1) {
            throw new ErrorApi(422, 'datos_invalidos', ['campo' => 'numero', 'motivo' => 'formato']);
        }
        $cuenta = Db::uno('SELECT id FROM cuentas WHERE numero_hmac = ?', [Seguridad::hmac($numero)]);
        if ($cuenta === null) throw new ErrorApi(404, 'no_existe');
        return new Respuesta(200, ['token' => self::nuevoToken($cuenta['id'])]);
    }

    /** Lo que se puede enseñar de la cuenta: correo enmascarado, si está verificado y cuándo se creó. */
    public static function ver(Solicitud $s): Respuesta
    {
        $f = Db::uno('SELECT correo_mascara, correo_verificado, creada FROM cuentas WHERE id = ?', [self::autenticar($s)]);
        return new Respuesta(200, [
            'correo'     => $f['correo_mascara'],
            'verificado' => $f['correo_verificado'] !== null,
            'creada'     => $f['creada'],
        ]);
    }

    /** Borra la cuenta y, en cascada, su progreso, sus sesiones y su historial. */
    public static function borrar(Solicitud $s): Respuesta
    {
        Db::ejecutar('DELETE FROM cuentas WHERE id = ?', [self::autenticar($s)]);
        return new Respuesta(204);
    }

    /** Un token nuevo de 256 bits para este dispositivo (180 días). Solo se guarda su huella. */
    public static function nuevoToken(string $cuentaId): string
    {
        $token = Seguridad::token();
        Db::ejecutar(
            'INSERT INTO tokens_dispositivo (token_hash, cuenta_id, creado, usado, caduca)
             VALUES (?, ?, UTC_TIMESTAMP(3), UTC_TIMESTAMP(3), UTC_TIMESTAMP(3) + INTERVAL 180 DAY)',
            [Seguridad::huella($token), $cuentaId],
        );
        return $token;
    }

    /** El id de la cuenta de este token, o 401. «Usada» se actualiza como mucho una vez al día. */
    public static function autenticar(Solicitud $s): string
    {
        $token = $s->token() ?? throw new ErrorApi(401, 'sin_sesion');
        $huella = Seguridad::huella($token);
        $fila = Db::uno(
            'SELECT cuenta_id FROM tokens_dispositivo WHERE token_hash = ? AND caduca > UTC_TIMESTAMP(3)',
            [$huella],
        ) ?? throw new ErrorApi(401, 'sin_sesion');
        Db::ejecutar(
            'UPDATE tokens_dispositivo SET usado = UTC_TIMESTAMP(3) WHERE token_hash = ? AND usado < UTC_TIMESTAMP(3) - INTERVAL 1 DAY',
            [$huella],
        );
        Db::ejecutar(
            'UPDATE cuentas SET usada = UTC_TIMESTAMP(3) WHERE id = ? AND usada < UTC_TIMESTAMP(3) - INTERVAL 1 DAY',
            [$fila['cuenta_id']],
        );
        return $fila['cuenta_id'];
    }
}
```

src/Progreso.php

```
<?php
// Progreso de cada curso: el mismo JSON que guarda la app (js/progress.js). Con versión: si otro dispositivo
// guardó antes, se responde 409 con lo que hay en la nube; la app lo junta y vuelve a guardar. Nadie pisa a nadie.
declare(strict_types=1);
```

```php
namespace Paglot;

final class Progreso
{
    private const PARTES = ['version', 'reiniciado', 'perfil', 'lecciones', 'cuaderno', 'actividad', 'extras'];

    public static function todos(Solicitud $s): Respuesta
    {
        $cuenta = Cuentas::autenticar($s);
        $cursos = [];
        foreach (Db::todos('SELECT curso, datos, version, actualizado FROM progreso WHERE cuenta_id = ?', [$cuenta]) as $f) {
            $cursos[$f['curso']] = self::fila($f);
        }
        return new Respuesta(200, ['cursos' => (object) $cursos]);
    }

    public static function leer(Solicitud $s, string $curso): Respuesta
    {
        $f = Db::uno(
            'SELECT datos, version, actualizado FROM progreso WHERE cuenta_id = ? AND curso = ?',
            [Cuentas::autenticar($s), $curso],
        );
        return new Respuesta(200, $f ? self::fila($f) : ['datos' => null, 'version' => 0, 'actualizado' => null]);
    }

    public static function guardar(Solicitud $s, string $curso): Respuesta
    {
        $cuenta = Cuentas::autenticar($s);
        $c = $s->cuerpo((int) Config::get('progreso_max') + 1024);
        $version = $c->version ?? null;
        if (!is_int($version) || $version < 0) {
            throw new ErrorApi(422, 'datos_invalidos', ['campo' => 'version', 'motivo' => 'tipo']);
        }
        $datos = self::validar($c->datos ?? null);
        $texto = json_encode($datos, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES | JSON_THROW_ON_ERROR);
        if (strlen($texto) > (int) Config::get('progreso_max')) throw new ErrorApi(413, 'demasiado_grande');

        return Db::transaccion(static function () use ($cuenta, $curso, $datos, $texto, $version): Respuesta {
            $f = Db::uno('SELECT datos, version FROM progreso WHERE cuenta_id = ? AND curso = ? FOR UPDATE', [$cuenta, $curso]);
            $actual = $f ? (int) $f['version'] : 0;
            if ($version !== $actual) {
                $copiaNube = $f ? json_decode($f['datos'], false, 512, JSON_THROW_ON_ERROR) : null;
                throw new ErrorApi(409, 'conflicto', ['datos' => $copiaNube, 'version' => $actual]);
            }
            if ($f === null) {
                Db::ejecutar(
                    'INSERT INTO progreso (cuenta_id, curso, datos, version, actualizado) VALUES (?, ?, ?, 1, UTC_TIMESTAMP(3))',
                    [$cuenta, $curso, $texto],
                );
                return new Respuesta(200, ['version' => 1]);
            }
            $antes = json_decode($f['datos'], false, 512, JSON_THROW_ON_ERROR);
            $reinicio = ($datos->reiniciado ?? 0) > ($antes->reiniciado ?? 0);
            if (!$reinicio && self::perdida($antes, $datos)) throw new ErrorApi(409, 'retroceso');

            // Copia de lo anterior como mucho una vez por hora (se guardan 14 días: lo limpia tareas/limpieza.php).
            $copiaReciente = Db::uno(
                'SELECT 1 AS si FROM progreso_historial WHERE cuenta_id = ? AND curso = ? AND guardado > UTC_TIMESTAMP(3) - INTERVAL 1 HOUR LIMIT 1',
                [$cuenta, $curso],
            );
            if ($copiaReciente === null) {
                Db::ejecutar(
                    'INSERT INTO progreso_historial (cuenta_id, curso, datos, version, guardado) VALUES (?, ?, ?, ?, UTC_TIMESTAMP(3))',
                    [$cuenta, $curso, $f['datos'], $actual],
                );
            }
            Db::ejecutar(
                'UPDATE progreso SET datos = ?, version = version + 1, actualizado = UTC_TIMESTAMP(3) WHERE cuenta_id = ? AND curso = ?',
                [$texto, $cuenta, $curso],
            );
            return new Respuesta(200, ['version' => $actual + 1]);
        });
    }

    public static function borrarCurso(Solicitud $s, string $curso): Respuesta
```

```
    {
        Db::ejecutar('DELETE FROM progreso WHERE cuenta_id = ? AND curso = ?', [Cuentas::autenticar($s), $curso]);
        return new Respuesta(204);
    }

    /** Si la copia nueva tiene menos de la mitad de lecciones y palabras que la guardada, algo ha fallado en la app. */
    private static function perdida(\stdClass $antes, \stdClass $ahora): bool
    {
        $total = static fn (\stdClass $d): int => count(get_object_vars($d->lecciones ?? new \stdClass()))
            + count(get_object_vars($d->cuaderno ?? new \stdClass()));
        return $total($antes) >= 10 && $total($ahora) * 2 < $total($antes);
    }

    /** Forma del progreso: versión 2, solo las partes conocidas y de su tipo. Lo demás se rechaza. */
    private static function validar(mixed $datos): \stdClass
    {
        if (!$datos instanceof \stdClass || ($datos->version ?? null) !== 2) {
            throw new ErrorApi(422, 'datos_invalidos', ['campo' => 'datos', 'motivo' => 'version']);
        }
        foreach (array_keys(get_object_vars($datos)) as $parte) {
            if (!in_array($parte, self::PARTES, true)) {
                throw new ErrorApi(422, 'datos_invalidos', ['campo' => "datos.$parte", 'motivo' => 'sobra']);
            }
        }
        foreach (['lecciones', 'cuaderno', 'actividad'] as $mapa) {
            if (isset($datos->$mapa) && !$datos->$mapa instanceof \stdClass) {
                throw new ErrorApi(422, 'datos_invalidos', ['campo' => "datos.$mapa", 'motivo' => 'tipo']);
            }
        }
        return $datos;
    }

    private static function fila(array $f): array
    {
        return [
            'datos'       => json_decode($f['datos'], false, 512, JSON_THROW_ON_ERROR),
            'version'     => (int) $f['version'],
            'actualizado' => $f['actualizado'],
        ];
    }
}
```

src/Correo.php

```
<?php
// Códigos de 6 cifras por correo (unir o recuperar) y envío de correos. Los códigos se guardan como HMAC,
// valen 10 minutos, solo vale el último pedido y se borran tras 5 fallos.
declare(strict_types=1);

namespace Paglot;

use PHPMailer\PHPMailer\PHPMailer;

final class Correo
{
    private static array $cola = [];

    /** Pide un código para unir un correo a la cuenta abierta. */
    public static function pedirVinculo(Solicitud $s): Respuesta
    {
        Cuentas::autenticar($s);
        $correo = self::correo($s->cuerpo(512));
        Limites::golpe('codigo_ip', $s->ip);
        Limites::golpe('codigo_correo', $correo);
        self::emitir($correo, 'vincular', 'Tu código para unir tu correo a Paglot');
        return new Respuesta(202, ['ok' => true]);
    }

    /** Comprueba el código y une el correo. Un correo ya unido a otra cuenta no se puede quitar: 409. */
    public static function confirmarVinculo(Solicitud $s): Respuesta
    {
        $cuenta = Cuentas::autenticar($s);
        $c = $s->cuerpo(512);
        $correo = self::correo($c);
        self::comprobarCodigo($correo, 'vincular', Solicitud::texto($c, 'codigo', 6));
        $hash = Seguridad::hmac($correo);
```

```
        if (Db::uno('SELECT id FROM cuentas WHERE correo_hash = ? AND id <> ?', [$hash, $cuenta]) !== null) {
            throw new ErrorApi(409, 'correo_en_uso');
        }
        Db::ejecutar(
            'UPDATE cuentas SET correo_hash = ?, correo_mascara = ?, correo_verificado = UTC_TIMESTAMP(3) WHERE id = ?',
            [$hash, Seguridad::enmascarar($correo), $cuenta],
        );
        return new Respuesta(200, ['correo' => Seguridad::enmascarar($correo)]);
    }

    /** Siempre 202, exista o no la cuenta: así no se puede saber qué correos tienen cuenta. */
    public static function pedirRecuperacion(Solicitud $s): Respuesta
    {
        $correo = self::correo($s->cuerpo(512));
        Limites::golpe('recuperar_ip', $s->ip);
        Limites::golpe('codigo_correo', $correo);
        $hay = Db::uno(
            'SELECT id FROM cuentas WHERE correo_hash = ? AND correo_verificado IS NOT NULL',
            [Seguridad::hmac($correo)],
        );
        if ($hay !== null) self::emitir($correo, 'recuperar', 'Tu código para recuperar Paglot');
        return new Respuesta(202, ['ok' => true]);
    }

    /** Con el código, abre una sesión nueva en la cuenta y cierra las anteriores. */
    public static function confirmarRecuperacion(Solicitud $s): Respuesta
    {
        $c = $s->cuerpo(512);
        $correo = self::correo($c);
        self::comprobarCodigo($correo, 'recuperar', Solicitud::texto($c, 'codigo', 6));
        $cuenta = Db::uno(
            'SELECT id FROM cuentas WHERE correo_hash = ? AND correo_verificado IS NOT NULL',
            [Seguridad::hmac($correo)],
        ) ?? throw new ErrorApi(400, 'codigo_invalido');
        Db::ejecutar('DELETE FROM tokens_dispositivo WHERE cuenta_id = ?', [$cuenta['id']]);
        return new Respuesta(200, ['token' => Cuentas::nuevoToken($cuenta['id'])]);
    }

    private static function correo(\stdClass $c): string
    {
        return Seguridad::correoNormal(Solicitud::texto($c, 'correo', 254))
            ?? throw new ErrorApi(422, 'datos_invalidos', ['campo' => 'correo', 'motivo' => 'formato']);
    }

    /** Guarda solo la huella del código (el último pedido es el único válido) y lo pone en cola para enviarlo. */
    private static function emitir(string $correo, string $accion, string $asunto): void
    {
        $codigo = Seguridad::codigo();
        Db::ejecutar(
            'INSERT INTO codigos (correo_hash, accion, huella, intentos, caduca)
             VALUES (?, ?, ?, 0, UTC_TIMESTAMP(3) + INTERVAL 10 MINUTE)
             ON DUPLICATE KEY UPDATE huella = VALUES(huella), intentos = 0, caduca = VALUES(caduca)',
            [Seguridad::hmac($correo), $accion, self::huellaCodigo($correo, $accion, $codigo)],
        );
        self::encolar($correo, $asunto, "Tu código es $codigo. Caduca en 10 minutos. Si no lo has pedido, ignora este correo.");
    }

    /**
     * Comprueba el código. El contador de fallos se guarda ANTES de responder con error: si se lanzara la excepción
     * dentro de la transacción, MariaDB la desharía y los intentos fallidos no contarían.
     */
    private static function comprobarCodigo(string $correo, string $accion, string $codigo): void
    {
        $hash = Seguridad::hmac($correo);
        $acierto = Db::transaccion(static function () use ($correo, $accion, $codigo, $hash): bool {
            $f = Db::uno(
                'SELECT huella, intentos FROM codigos WHERE correo_hash = ? AND accion = ? AND caduca > UTC_TIMESTAMP(3) FOR UPDATE',
                [$hash, $accion],
            );
            if ($f === null) return false;
            if (!hash_equals((string) $f['huella'], self::huellaCodigo($correo, $accion, $codigo))) {
                Db::ejecutar('UPDATE codigos SET intentos = intentos + 1 WHERE correo_hash = ? AND accion = ?', [$hash, $accion]);
                Db::ejecutar('DELETE FROM codigos WHERE correo_hash = ? AND accion = ? AND intentos >= 5', [$hash, $accion]);
                return false;
            }
            Db::ejecutar('DELETE FROM codigos WHERE correo_hash = ? AND accion = ?', [$hash, $accion]);   // un solo uso
```

```
            return true;
        });
        if (!$acierto) throw new ErrorApi(400, 'codigo_invalido');
    }

    private static function huellaCodigo(string $correo, string $accion, string $codigo): string
    {
        return hash_hmac('sha256', $accion . '|' . $correo . '|' . $codigo, (string) Config::get('secreto'), true);
    }

    /** Pone un correo en cola. Se envía después de responder (ver public/index.php). */
    public static function encolar(string $para, string $asunto, string $texto, ?string $responder = null): void
    {
        self::$cola[] = compact('para', 'asunto', 'texto', 'responder');
    }

    public static function enviarEncolados(): void
    {
        foreach (self::$cola as $mensaje) {
            try {
                self::mandar($mensaje);
            } catch (\Throwable $e) {
                // Sin destinatario ni contenido en el registro. En producción, aquí va una cola de reintentos.
                error_log('[paglot] correo no enviado: ' . $e::class);
            }
        }
        self::$cola = [];
    }

    private static function mandar(array $m): void
    {
        if (Config::get('correo.modo') === 'archivo') {    // pruebas en el ordenador: el correo queda como archivo
            file_put_contents(
                Config::get('correo.buzon') . '/' . bin2hex(random_bytes(6)) . '.json',
                json_encode($m, JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR),
            );
            return;
        }
        $p = new PHPMailer(true);
        $p->isSMTP();
        $p->Host = (string) Config::get('correo.smtp.host');
        $p->Port = (int) Config::get('correo.smtp.puerto');
        $p->SMTPAuth = true;
        $p->Username = (string) Config::get('correo.smtp.usuario');
        $p->Password = (string) Config::get('correo.smtp.contrasena');
        $p->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
        $p->CharSet = 'UTF-8';
        $p->setFrom((string) Config::get('correo.desde'), 'Paglot');
        $p->addAddress($m['para']);
        if ($m['responder'] !== null) $p->addReplyTo($m['responder']);
        $p->Subject = $m['asunto'];
        $p->Body = $m['texto'];
        $p->send();
    }
}
```

src/Errores.php

```
<?php
// Informes de errores anónimos: sin número de cuenta, sin correo y sin lo que la persona ha escrito.
declare(strict_types=1);

namespace Paglot;

final class Errores
{
    public static function registrar(Solicitud $s): Respuesta
    {
        Limites::golpe('errores', $s->ip);
        $c = $s->cuerpo(4096);
        $mensaje = self::limpiar(Solicitud::texto($c, 'mensaje', 600));
        if ($mensaje === '') return new Respuesta(204);
        $donde     = self::limpiar(Solicitud::texto($c, 'donde', 200, false) ?? '');
        $pantalla  = mb_substr(explode('?', self::limpiar(Solicitud::texto($c, 'pantalla', 200, false) ?? ''))[0], 0, 80);
        $curso     = mb_substr((string) preg_replace('/[^a-z]/', '', Solicitud::texto($c, 'curso', 20, false) ?? ''), 0, 8);
        $version   = mb_substr(Solicitud::texto($c, 'version', 40, false) ?? '', 0, 40);
```

```
        $navegador = mb_substr(self::limpiar(Solicitud::texto($c, 'navegador', 80, false) ?? ''), 0, 60);
        $mensaje   = mb_substr($mensaje, 0, 300);
        $huella    = hash('sha256', gmdate('Y-m-d') . "|$mensaje|$donde|$version|$navegador", true);

        // Los iguales del mismo día se cuentan en una sola fila.
        Db::ejecutar(
            'INSERT INTO errores (huella, dia, mensaje, donde, pantalla, curso, version, navegador, veces, primera, ultima)
             VALUES (?, UTC_DATE(), ?, ?, ?, ?, ?, ?, 1, UTC_TIMESTAMP(3), UTC_TIMESTAMP(3))
             ON DUPLICATE KEY UPDATE veces = veces + 1, ultima = UTC_TIMESTAMP(3)',
            [$huella, $mensaje, $donde, $pantalla, $curso, $version, $navegador],
        );
        return new Respuesta(204);
    }

    /** Quita lo que parece personal: correos y números largos (cuentas, teléfonos). */
    private static function limpiar(string $texto): string
    {
        $limpio = (string) preg_replace(['/[^\s@]+@[^\s@]+\.[a-z]{2,}/i', '/\d{6,}/'], '[quitado]', trim($texto));
        return mb_substr($limpio, 0, 600);
    }
}
```

src/Ideas.php

```
<?php
// Laboratorio de ideas: todos ven las propuestas; proponer, votar y denunciar exige cuenta (y así cuesta más hacer trampa).
declare(strict_types=1);

namespace Paglot;

final class Ideas
{
    public static function listar(Solicitud $s): Respuesta
    {
        $cuenta = $s->token() !== null ? Cuentas::autenticar($s) : null;
        $filas = Db::todos(
            'SELECT i.id, i.texto, i.autora, i.creada,
                    (SELECT COUNT(*) FROM ideas_votos v WHERE v.idea_id = i.id) AS votos,
                    EXISTS (SELECT 1 FROM ideas_votos w WHERE w.idea_id = i.id AND w.cuenta_id = ?) AS votada
             FROM ideas i
             WHERE i.oculta = 0
             ORDER BY votos DESC, i.creada
             LIMIT 200',
            [$cuenta],
        );
        return new Respuesta(200, ['ideas' => array_map(static fn (array $f): array => [
            'id'     => $f['id'],
            'texto'  => $f['texto'],
            'autora' => $f['autora'],
            'votos'  => (int) $f['votos'],
            'votada' => (bool) $f['votada'],
            'creada' => $f['creada'],
        ], $filas)]);
    }

    public static function nueva(Solicitud $s): Respuesta
    {
        $cuenta = Cuentas::autenticar($s);
        Limites::golpe('idea', $cuenta);   // tres al día por cuenta
        $c = $s->cuerpo(2048);
        $texto = trim(Solicitud::texto($c, 'texto', 180));
        $autora = trim(Solicitud::texto($c, 'autora', 32, false) ?? '');
        if (mb_strlen($texto) < 3) throw new ErrorApi(422, 'datos_invalidos', ['campo' => 'texto', 'motivo' => 'corto']);
        if (self::tieneEnlace("$texto $autora")) throw new ErrorApi(422, 'enlace');
        $id = Seguridad::uuid();
        Db::ejecutar(
            'INSERT INTO ideas (id, texto, autora, cuenta_id, creada) VALUES (?, ?, ?, ?, UTC_TIMESTAMP(3))',
            [$id, $texto, $autora, $cuenta],
        );
        return new Respuesta(201, ['id' => $id]);
    }

    /** Filtro sencillo (web y dominios habituales). No es perfecto: la denuncia y la revisión son la red de seguridad. */
    private static function tieneEnlace(string $t): bool
    {
        return preg_match('#https?://|www\.|\b[a-z0-9-]+\.(com|net|org|io|es|app|me|ly|xyz)\b#i', $t) === 1;
```

```
    }

    public static function votar(Solicitud $s, string $id): Respuesta
    {
        $cuenta = Cuentas::autenticar($s);
        Limites::golpe('voto', $cuenta);
        self::existe($id);
        Db::ejecutar('INSERT IGNORE INTO ideas_votos (idea_id, cuenta_id) VALUES (?, ?)', [$id, $cuenta]);
        return new Respuesta(204);
    }

    public static function quitarVoto(Solicitud $s, string $id): Respuesta
    {
        Db::ejecutar('DELETE FROM ideas_votos WHERE idea_id = ? AND cuenta_id = ?', [$id, Cuentas::autenticar($s)]);
        return new Respuesta(204);
    }

    /** Tres cuentas distintas que denuncian ocultan la idea hasta que alguien la revise. */
    public static function denunciar(Solicitud $s, string $id): Respuesta
    {
        $cuenta = Cuentas::autenticar($s);
        Limites::golpe('denuncia', $cuenta);
        self::existe($id);
        Db::transaccion(static function () use ($id, $cuenta): void {
            Db::ejecutar('INSERT IGNORE INTO ideas_denuncias (idea_id, cuenta_id) VALUES (?, ?)', [$id, $cuenta]);
            $n = (int) Db::uno('SELECT COUNT(*) AS n FROM ideas_denuncias WHERE idea_id = ?', [$id])['n'];
            if ($n >= 3) Db::ejecutar('UPDATE ideas SET oculta = 1 WHERE id = ?', [$id]);
        });
        return new Respuesta(204);
    }

    private static function existe(string $id): void
    {
        if (Db::uno('SELECT id FROM ideas WHERE id = ? AND oculta = 0', [$id]) === null) {
            throw new ErrorApi(404, 'no_encontrado');
        }
    }
}
```

src/Contacto.php

```
<?php
// Formulario de contacto: un correo al equipo, con la respuesta dirigida a la persona. No se guarda nada en la base.
declare(strict_types=1);

namespace Paglot;

final class Contacto
{
    private const ASUNTOS = ['general', 'tecnico', 'lecciones', 'pagos', 'escuelas', 'idea', 'privacidad'];

    public static function enviar(Solicitud $s): Respuesta
    {
        Limites::golpe('contacto', $s->ip);
        $c = $s->cuerpo(16384);
        // Campo trampa: las personas no lo ven; los robots que lo rellenan reciben un «ok» y no se envía nada.
        if (($c->web ?? '') !== '') return new Respuesta(202, ['ok' => true]);

        self::turnstile(Solicitud::texto($c, 'turnstile', 2048), $s->ip);
        $correo = Seguridad::correoNormal(Solicitud::texto($c, 'correo', 254))
            ?? throw new ErrorApi(422, 'datos_invalidos', ['campo' => 'correo', 'motivo' => 'formato']);
        $asunto = Solicitud::texto($c, 'asunto', 20);
        if (!in_array($asunto, self::ASUNTOS, true)) {
            throw new ErrorApi(422, 'datos_invalidos', ['campo' => 'asunto', 'motivo' => 'valor']);
        }
        $mensaje = trim(Solicitud::texto($c, 'mensaje', 4000));
        if (mb_strlen($mensaje) < 5) throw new ErrorApi(422, 'datos_invalidos', ['campo' => 'mensaje', 'motivo' => 'corto']);
        $nombre = str_replace(["\r", "\n"], ' ', mb_substr(Solicitud::texto($c, 'usuario', 60, false) ?? '', 0, 60));

        Correo::encolar(
            (string) Config::get('correo.equipo'),
            "[Paglot contacto · $asunto]",
            "De: $nombre <$correo>\n\n$mensaje",
            $correo,
        );
        return new Respuesta(202, ['ok' => true]);
```

```
    }

    /** Cloudflare Turnstile: la comprobación la hace el servidor, nunca la propia página. */
    private static function turnstile(string $token, string $ip): void
    {
        $secreto = (string) Config::get('turnstile.secreto');
        if ($secreto === '') return;   // sin clave (pruebas en el ordenador): solo cuentan el límite y el campo trampa
        $ch = curl_init('https://challenges.cloudflare.com/turnstile/v0/siteverify');
        curl_setopt_array($ch, [
            CURLOPT_POST           => true,
            CURLOPT_POSTFIELDS     => ['secret' => $secreto, 'response' => $token, 'remoteip' => $ip],
            CURLOPT_RETURNTRANSFER => true,
            CURLOPT_TIMEOUT        => 5,
        ]);
        $respuesta = json_decode((string) curl_exec($ch), true);
        if (!is_array($respuesta) || ($respuesta['success'] ?? false) !== true) {
            throw new ErrorApi(422, 'comprobacion_fallida');
        }
    }
}
```

`tareas/limpieza.php`

```
<?php
// Cron cada noche (03:30 UTC): borra lo caducado. Usa el usuario de tareas, no el de la API.
//   php tareas/limpieza.php
declare(strict_types=1);

if (PHP_SAPI !== 'cli') {
    http_response_code(404);
    exit;
}
require dirname(__DIR__) . '/src/Arranque.php';

use Paglot\Db;

Db::comoTareas();
$pasos = [
    'tokens de dispositivo caducados'    => 'DELETE FROM tokens_dispositivo WHERE caduca < UTC_TIMESTAMP(3)',
    'códigos caducados'                  => 'DELETE FROM codigos WHERE caduca < UTC_TIMESTAMP(3)',
    'límites viejos'                     => 'DELETE FROM limites WHERE inicio < UTC_TIMESTAMP(3) - INTERVAL 2 DAY',
    'copias de progreso (14 días)'       => 'DELETE FROM progreso_historial WHERE guardado < UTC_TIMESTAMP(3) - INTERVAL 14 DAY',
    'cuentas anónimas sin usar (7 días)' => 'DELETE FROM cuentas WHERE correo_verificado IS NULL AND usada < UTC_TIMESTAMP(3) - INTERVAL 7 DAY',
    'errores (90 días)'                  => 'DELETE FROM errores WHERE ultima < UTC_TIMESTAMP(3) - INTERVAL 90 DAY',
];

$total = 0;
foreach ($pasos as $nombre => $sql) {
    $borradas = Db::ejecutar($sql);
    $total += $borradas;
    printf("%-38s %d\n", $nombre, $borradas);
}
printf("Limpieza terminada: %d filas.\n", $total);
```

La configuración de Nginx y el script de copias están en el capítulo 9; el cliente de la app, en el apartado 6.8.

# 6. Cómo probarlo en local en Windows

## 6.1 Lo que tienes que instalar tú

El asistente no descarga nada: las descargas las haces tú, de las fuentes oficiales.

- **PHP 8.4** (versión *Thread Safe* del zip de windows.php.net). Alternativa: **Laragon**, que trae PHP y MariaDB (no verificado hoy).
- **MariaDB 11.8 LTS** (instalador MSI de mariadb.org), o el de Laragon.
- **Composer** (getcomposer.org), para PHPMailer.
- En `php.ini` hay que activar: `extension=pdo_mysql`, `extension=mbstring`, `extension=curl` y `extension=openssl`.

## 6.2 Base de datos

Desde PowerShell, dentro de la carpeta `api\`. Ajusta la ruta de `mariadb.exe` a tu instalación. Antes de ejecutar `permisos.sql`, cambia sus contraseñas.

```
$m = "C:\Program Files\MariaDB 11.8\bin\mariadb.exe"
& $m -u root -p -e "CREATE DATABASE IF NOT EXISTS paglot CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
Get-Content sql\esquema.sql | & $m -u root -p paglot
Get-Content sql\permisos.sql | & $m -u root -p
```

## 6.3 Configuración

```
$env:PAGLOT_ENTORNO = 'local'
$env:PAGLOT_SECRETO = php -r "echo bin2hex(random_bytes(32));"
$env:PAGLOT_BD_CONTRASENA = 'la del usuario paglot_app'
$env:PAGLOT_BD_CONTRASENA_TAREAS = 'la del usuario paglot_tareas'
$env:PAGLOT_CORREO_MODO = 'archivo'        # los correos quedan en buzon\ en vez de enviarse
New-Item -ItemType Directory -Force buzon | Out-Null
Copy-Item config\config.ejemplo.php config\config.php
composer require phpmailer/phpmailer
```

El secreto (`PAGLOT_SECRETO`) debe ser el **mismo** cada vez que se arranque, o las huellas de la base ya no encajarán. Guárdalo en tu gestor de contraseñas.

## 6.4 Arrancar la API

```
php -S 127.0.0.1:8180 -t public public/index.php
```

El servidor integrado de PHP manda todas las peticiones a `public/index.php`, igual que Nginx en el servidor real.

## 6.5 Pruebas con curl (en Git Bash, o en WSL)

Las pruebas van en orden: cada paso usa lo que deja el anterior. Requieren Python para leer el JSON (ya está en el equipo).

```
API=http://127.0.0.1:8180/api/v1
J='Content-Type: application/json'

# 1. Salud
curl -s -w ' [%{http_code}]\n' $API/salud

# 2. Crear cuenta: se guardan el número y el token
RESP=$(curl -s -X POST $API/cuentas -H "$J" -d '{}')
echo "$RESP"
TOKEN=$(echo "$RESP" | python -c "import sys,json;print(json.load(sys.stdin)['token'])")
NUMERO=$(echo "$RESP" | python -c "import sys,json;print(json.load(sys.stdin)['numero'])")

# 3. Primera escritura del curso «es» (versión 0) → 200 {version: 1}
DATOS='{"version":2,"perfil":{"nombre":"Ana","t":1},"lecciones":{},"cuaderno":{},"actividad":{}}'
curl -s -w ' [%{http_code}]\n' -X PUT $API/progreso/es -H "$J" -H "Authorization: Bearer $TOKEN" -d "{\"datos\":$DATOS,\"version\":0}"

# 4. Otra escritura con la versión vieja (0) → 409 conflicto, con la copia que hay en la nube
curl -s -w ' [%{http_code}]\n' -X PUT $API/progreso/es -H "$J" -H "Authorization: Bearer $TOKEN" -d "{\"datos\":$DATOS,\"version\":0}"

# 5. Sin token → 401
curl -s -w ' [%{http_code}]\n' $API/progreso

# 6. Origen que no está en la lista → 403
curl -s -w ' [%{http_code}]\n' $API/salud -H 'Origin: https://malo.example'

# 7. Preflight desde el APK (Android) → 204 con Access-Control-Allow-Origin
curl -s -i -X OPTIONS $API/progreso/es -H 'Origin: https://localhost' -H 'Access-Control-Request-Method: PUT' | head -n 12

# 8. Entrar con número: formato incorrecto (422) y número que no existe (404)
curl -s -w ' [%{http_code}]\n' -X POST $API/cuentas/entrar -H "$J" -d '{"numero":"123"}'
curl -s -w ' [%{http_code}]\n' -X POST $API/cuentas/entrar -H "$J" -d '{"numero":"9999999999999999"}'

# 9. Límite de entrar: 20 por hora y IP. Como el paso 8 ya contó 2, el 429 aparece hacia la 19.ª de este bucle
for i in $(seq 1 25); do curl -s -o /dev/null -w "%{http_code} " -X POST $API/cuentas/entrar -H "$J" -d '{"numero":"8888888888888888"}'; done; echo

# 10. Código de correo: se pide con sesión y el código aparece en buzon/ (archivo .json)
curl -s -w ' [%{http_code}]\n' -X POST $API/cuenta/correo -H "$J" -H "Authorization: Bearer $TOKEN" -d '{"correo":"ana@example.com"}'
CODIGO=$(python -c "import json,glob,os,re;f=max(glob.glob('buzon/*.json'),key=os.path.getmtime);print(re.search(r'\d{6}',json.load(open(f,encoding='utf-8'))['t

# 10b. Cinco fallos seguidos: 400 cada vez; al quinto se borra el código
for i in 1 2 3 4 5; do curl -s -o /dev/null -w "%{http_code} " -X POST $API/cuenta/correo/confirmar -H "$J" -H "Authorization: Bearer $TOKEN" -d '{"correo":"ana
# 10c. Ahora ni el código bueno vale (ya se ha gastado): 400
curl -s -w ' [%{http_code}]\n' -X POST $API/cuenta/correo/confirmar -H "$J" -H "Authorization: Bearer $TOKEN" -d "{\"correo\":\"ana@example.com\",\"codigo\":\"$

# 11. Recuperación: 202 exista o no el correo
curl -s -w ' [%{http_code}]\n' -X POST $API/recuperar -H "$J" -d '{"correo":"nadie@example.com"}'

# 12. Errores: el correo y el número largo salen como [quitado] en la base
curl -s -w ' [%{http_code}]\n' -X POST $API/errores -H "$J" -d '{"mensaje":"Fallo con ana@example.com y 1234567890123456","donde":"js/app.js:10:5","pantalla":"#

# 13. Ideas: un enlace en el texto se rechaza (422)
curl -s -w ' [%{http_code}]\n' -X POST $API/ideas -H "$J" -H "Authorization: Bearer $TOKEN" -d '{"texto":"Mirad www.ejemplo.com"}'
```

```
# 14. Contacto: con el campo trampa relleno → 202 pero no sale ningún correo; sin trampa → 202 y un archivo en buzon\
curl -s -w ' [%{http_code}]\n' -X POST $API/contacto -H "$J" -d '{"correo":"ana@example.com","asunto":"general","mensaje":"Hola, prueba","turnstile":"x","web":"
curl -s -w ' [%{http_code}]\n' -X POST $API/contacto -H "$J" -d '{"correo":"ana@example.com","asunto":"general","mensaje":"Hola, prueba","turnstile":"x","web":"

# 15. Limpieza nocturna con el usuario de tareas
php tareas/limpieza.php
```

**Resultados esperados.** Paso 2: `201`. Paso 3: `200` y `{"version":1}`. Paso 4: `409` con `"error":"conflicto"`, `"version":1` y la copia. Paso 5: `401`. Paso 6: `403`. Paso 7: `204` con `Access-Control-Allow-Origin: https://localhost`. Paso 8: `422` y `404`. Paso 9: `429` a partir de la 19.ª petición, con `Retry-After`. Paso 10: `202`. Paso 10b: `400` cinco veces. Paso 10c: `400`. Paso 11: `202`. Paso 12: `204`, y en la base, `[quitado]` en lugar del correo y del número. Paso 13: `422` con `"error":"enlace"`. Paso 14: `202` en los dos casos, pero solo el segundo deja un archivo en `buzon\`.

## 6.6 Prueba de dos dispositivos (la que evita perder datos)

Es la prueba más importante, porque es la que falla hoy (B3):

1. Dispositivo A y dispositivo B entran con el mismo número (dos llamadas a `/cuentas/entrar`, dos tokens).
2. A lee el curso (`GET /progreso/es`, versión 1).
3. B escribe un cambio (`PUT`, versión 1 → 2).
4. A escribe su cambio con la versión 1 → **409** con la copia de B.
5. A junta las dos copias con `unirDatos` y vuelve a escribir con la versión 2 → **200**.

Resultado correcto: la copia final tiene los cambios de A y de B. Con la app de hoy, el paso 4 no existe: A escribe directamente y el cambio de B se pierde.

## 6.7 Pruebas que faltan

- **Pruebas unitarias** (PHPUnit): `Seguridad`, `Limites` y la función `perdida()` de `Progreso`.
- **Concurrencia**: 20 `PUT` simultáneos con la misma versión deben dar **un** 200 y 19 409. Se puede hacer con `xargs -P 20` en Git Bash.
- **Carga**: unas 200 peticiones por segundo durante cinco minutos, con una herramienta como `hey` o `k6` (no verificadas hoy).
- **Restauración**: restaurar una copia de la base en otra máquina cada mes. Una copia que nunca se ha restaurado no sirve.
- **Seguridad**: un escaneo de línea base contra el entorno de pruebas (OWASP ZAP o similar; no verificado hoy).
- **CORS desde iOS**: repetir el paso 7 con `Origin: capacitor://localhost`.

## 6.8 Cambios en la app para probarla contra la API local

1. `js/config.js`: `export const API_URL = 'http://127.0.0.1:8180/api/v1';` (solo para pruebas; en producción, `https://api.paglot.app/api/v1`).
2. `index.html`, Content-Security-Policy: `connect-src 'self' http://127.0.0.1:8180 https://api.paglot.app`. Al migrar, quitar la URL de Supabase.
3. `js/nube.js`: sustituir `rpc()` y `auth()` por las llamadas de este ejemplo y guardar el token junto al número (por ejemplo, en `paglot-token`). Un token en `localStorage` queda al alcance del JavaScript de la página; como la app no pinta datos de la nube con `innerHTML`, el riesgo es el de siempre de una app web, pero conviene recordarlo.
4. `sw.js`: revisar que la caché del service worker no sirva respuestas antiguas de la API (no verificado hoy).

Cliente de ejemplo (patrón de las llamadas, con la fusión en caso de 409):

`cliente/nube-api.js (patrón para js/nube.js)`

```
// Patrón para js/nube.js con la API propia. No es un archivo que se ejecute tal cual: es la forma de las llamadas
// que sustituyen a rpc() de Supabase. El número y el token se guardan en el dispositivo; el número ya no sirve
// como llave en cada petición, solo para entrar en otro dispositivo.
import { API_URL } from './config.js';
import { unirDatos } from './progress.js';

async function api(ruta, { metodo = 'GET', token = null, cuerpo = null } = {}) {
  const respuesta = await fetch(`${API_URL}${ruta}`, {
    method: metodo,
    headers: {
      ...(cuerpo ? { 'Content-Type': 'application/json' } : {}),
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
    },
    body: cuerpo ? JSON.stringify(cuerpo) : undefined,
    signal: AbortSignal.timeout(8000),
  });
  const datos = respuesta.status === 204 ? null : await respuesta.json().catch(() => null);
  if (!respuesta.ok) throw Object.assign(new Error(datos?.error ?? `HTTP ${respuesta.status}`), { estado: respuesta.status, datos });
  return datos;
}

export const crearCuenta = () => api('/cuentas', { metodo: 'POST', cuerpo: {} });            // { numero, token }
export const entrarConNumero = numero => api('/cuentas/entrar', { metodo: 'POST', cuerpo: { numero } }); // { token }
export const bajarProgreso = (token, curso) => api(`/progreso/${curso}`, { token });          // { datos, version }

/** Guarda el progreso de un curso. Si otro dispositivo guardó antes (409), se junta y se vuelve a guardar. */
export async function guardarProgreso(token, curso, copia, version, intento = 0) {
  try {
    const { version: nueva } = await api(`/progreso/${curso}`, { metodo: 'PUT', token, cuerpo: { datos: copia, version } });
    return nueva;
  } catch (error) {
```

```
    if (error.datos?.error !== 'conflicto' || intento >= 3) throw error;
    // Las partes de «extras» (estrellas, cofres, logros) se juntan igual que en importarCopia() de progress.js.
    const juntado = error.datos.datos ? unirDatos(copia, error.datos.datos) : copia;
    return guardarProgreso(token, curso, juntado, error.datos.version, intento + 1);
  }
}
```

## 7. Migración desde Supabase: exportar, transformar, importar

### 7.1 Principios

1. **Ningún número cambia.** Las APK instaladas guardan el número; si el servidor nuevo lo acepta igual, nadie pierde la cuenta.
2. **Nada se escribe en dos sitios a la vez** sin necesidad. Escribir en los dos es posible, pero complica todo y da más fallos. Se prefiere una ventana corta de congelación.
3. **Las APK antiguas siguen funcionando** durante meses. Un usuario que no actualiza no debe perder datos ni cuenta.
4. **Siempre hay vuelta atrás** durante un periodo, y se prueba antes de hacer nada en serio.

### 7.2 Qué se migra y qué no

| Origen (Supabase) | Destino (API) | Criterio |
|---|---|---|
| `paglot_cuentas` con `usuario` | `cuentas` (con `correo_hash`, `correo_mascara`, `correo_verificado`) | Todas |
| `paglot_cuentas` sin `usuario` y `usada` de hace más de 7 días | — | No se migran: ya habrían caducado |
| `paglot_cuentas` sin `usuario` y `usada` reciente | `cuentas` | Sí |
| `datos.cursos[<curso>]` (versión 2 de la copia) | `progreso` (una fila por curso, `version = 1`) | Cada curso |
| `datos` con la forma antigua (sin `cursos`, versión 2) | `progreso` con el curso `es` | Solo en las cuentas de cuando la app era de español |
| `auth.users` del correo de cada cuenta | Solo el correo enmascarado y la huella | El usuario de Auth se queda en Supabase y se borra al cerrar |
| `paglot_errores` | — | No: son de hace menos de 90 días y no tienen valor |
| `paglot_ideas`, `paglot_votos`, `paglot_denuncias` | `ideas`, `ideas_votos`, `ideas_denuncias` | Solo si se han aplicado en Supabase. Los votos de marca aleatoria no se convierten |
| Códigos de correo pendientes | — | No: caducan en 10 minutos |

### 7.3 Exportar desde Supabase

Antes de empezar, exporta **todo** y guarda el archivo fuera de tu equipo. Para exportar, usa `psql` con la cadena de conexión del panel de Supabase (no la pegues en ningún sitio que se comparta), y el rol `postgres`:

```
-- Cuentas (\copy de psql: lo pones en la consola de psql, no en el editor)
\copy (select numero, datos, usuario, correo, creada, usada, escrita from public.paglot_cuentas) to 'cuentas.csv' csv header

-- Correos verificados de Auth (solo los que tienen cuenta)
\copy (select c.numero, u.id as auth_id, u.email, u.email_confirmed_at from public.paglot_cuentas c join auth.users u on u.id = c.usuario) to 'correos.csv' csv

-- Ideas y votos, si se han aplicado
\copy (select * from public.paglot_ideas) to 'ideas.csv' csv header
\copy (select * from public.paglot_votos) to 'votos.csv' csv header
```

Los ficheros contienen datos personales: tenlos en una carpeta cifrada o bórralos al terminar la migración.

### 7.4 Transformar

Reglas que el importador aplica (se describen aquí; el importador se escribe cuando se decida migrar):

- **Número**: `numero_hmac = HMAC-SHA256(numero, PAGLOT_SECRETO)`, igual que la API. El número en claro no se guarda.
- **Cuenta**: `id` nuevo (UUID v4). `creada` y `usada` se conservan.
- **Correo** (si `usuario` no es nulo): `correo_hash = HMAC(correo en minúsculas)`, `correo_mascara` y `correo_verificado = email_confirmed_at`. Si `correo_hash` ya existe en otra cuenta (no debería), la fila se revisa a mano.
- **Progreso**: por cada `cursos.<c>` que tenga `version = 2`, una fila de `progreso` con esa copia y `version = 1`. Si la copia no tiene `version = 2`, no se importa y se apunta en un registro.
- **Validación**: el JSON de cada curso se comprueba con la misma regla de `Progreso::validar()` (versión 2 y partes conocidas). Lo que no pasa se apunta, no se descarta en silencio.
- **Idempotencia**: el importador usa `INSERT … ON DUPLICATE KEY UPDATE` por `numero_hmac`. Así se puede ejecutar varias veces (por ejemplo, para la carga final).

### 7.5 Importar

Un script de línea de comandos (`tareas/importar_supabase.php`) lee los CSV, aplica las reglas anteriores en lotes de 500 filas dentro de transacciones y escribe un resumen: filas leídas, filas importadas, filas rechazadas y el motivo de cada rechazo.

### 7.6 Verificar antes de dar nada por bueno

1. **Recuentos**: el número de cuentas y de cursos en el destino es igual al del origen menos los descartes (anónimas caducadas, errores, códigos).
2. **Muestra**: para 50 cuentas elegidas al azar, se entra con su número en la API (`/cuentas/entrar`), se lee `/progreso` y se compara el JSON con el de Supabase. Deben ser idénticos.

3. **Tamaños**: suma de bytes de `progreso.datos` por curso, para comprobar el límite de 2 MB de la copia antigua.
4. **Correos**: los enmascarados coinciden con los del origen, y ninguno aparece en claro en la base de destino (`SELECT` sobre la tabla, buscando `@`).

### 7.7 Plan por fases, sin pérdidas

| Fase | Cuándo | Qué se hace | Vuelta atrás |
|---|---|---|---|
| 0. Preparación | Semanas antes | API en un VPS de pruebas; importación de prueba con una copia de los datos; verificación (7.6) | No hay nada que deshacer |
| 1. Aviso | 7 días antes | Aviso en la app (y en la web) de que habrá una pausa de unos minutos | — |
| 2. Congelación | El día del corte, a primera hora | En Supabase se quita la posibilidad de escribir: `revoke execute on function public.paglot_guardar(text, jsonb) from public, anon, authenticated;` (lo mismo con `paglot_borrar`, `paglot_vincular` y `paglot_idea_nueva`, si están). Hay que quitarla también de `PUBLIC`, porque `anon` la hereda por defecto. Las lecturas siguen funcionando. Probarlo antes en el proyecto de pruebas | `grant execute on function … to public, anon, authenticated;` (con los mismos nombres) |
| 3. Carga final | Mismo día | Export de lo que haya cambiado desde la fase 0 (filas con `escrita` posterior), importación idempotente y verificación (7.6) | Se repite la importación |
| 4. Corte | Cuando la verificación da bien | Se publica la web y el APK nuevos, con `API_URL` apuntando a la API | Se vuelve a publicar la versión anterior de la web; el APK antiguo sigue en Supabase |
| 5. Adaptador | Durante meses | Ver 7.8 | Quitarlo cuando ya no haya APK antiguas en uso |
| 6. Vigilancia | 90 días | Supabase en solo lectura, sin borrar nada; se vigilan errores y tráfico | Reactivar escrituras |
| 7. Retirada | Después de 90 días y revisado el RGPD | Exportación final y cierre del proyecto de Supabase | — |

**Tiempo de congelación.** Con la carga final de unos minutos, la pausa visible es de media hora o menos (estimación). Hay que decirlo en el aviso.

**Qué pasa con quien tiene la APK antigua durante la congelación.** Al intentar guardar, verá un error («sin conexión» o error de servidor, según el código de la app). Sus datos siguen en su móvil y se suben cuando se restablezca el servicio o se actualice la app.

### 7.8 El adaptador para las APK antiguas

Las APK ya instaladas llaman a `https://iuniglsiqnfvnotmfgdo.supabase.co/rest/v1/rpc/…` y a `auth/v1/…`. Si Supabase se cierra, esas APK dejan de sincronizar. Hay dos caminos:

- **Adaptador (recomendado para los primeros meses).** Una ruta en la API que reciba las mismas peticiones que hoy (`/rest/v1/rpc/paglot_leer`, `paglot_guardar`, `paglot_existe`…) y las traduzca a la lógica nueva, con la misma forma de respuesta. Son unas 10 rutas pequeñas. Es trabajo de 1 o 2 días (estimación), y evita que nadie pierda datos.
- **Congelación prolongada.** Más barato, pero los usuarios de APK antiguas no pueden sincronizar hasta actualizar.

El adaptador recibe el número, como hacía Supabase, y para cada petición obtiene internamente un token con `/cuentas/entrar`, lo usa y lo descarta. No guarda nada. Es el punto más delicado: el número sigue viajando en cada petición de esas APK antiguas (es lo que tienen). Por eso el adaptador tiene el mismo límite de intentos que la API y conviene probarlo con una APK antigua en un móvil real antes del corte.

### 7.9 Plan de vuelta atrás

Si algo sale mal en las primeras 24 horas:

1. Volver a publicar la web anterior (apuntando a Supabase).
2. Volver a dar permisos de escritura en Supabase (`grant`).
3. Los datos que se hayan escrito en la API después del corte **no** están en Supabase. Hay que exportarlos (hay un script para ese sentido, a escribir) y reimportarlos en Supabase. Por eso, mientras se decide, la API se pone en modo de solo lectura.

No se recomienda escribir en los dos sistemas a la vez: la vuelta atrás se complica mucho y el riesgo de que dos copias divergan crece con cada usuario.

## 8. Hostings, correo, dominios y recomendación

Estado de cada cifra: **verificado** (página oficial abierta el 19-10-2026), **resultado de búsqueda** (fuente secundaria o página que no se pudo abrir), o **estimación**. Precios en euros sin IVA, salvo que se indique. Las fuentes están en el capítulo 10.

### 8.1 Alojamiento compartido (España y UE)

Para la API es una opción **solo si se confirma por escrito** lo que falta en la tabla: PHP 8.3 o superior a elegir, MariaDB o MySQL, SSH, cron y centro de datos en la UE. Sin SSH ni cron no se puede mantener la API ni las copias de forma seria.

| Proveedor | Plan y precio | Estado del precio | PHP / MySQL / SSH / cron | Centro de datos | Comentario |
|---|---|---|---|---|---|
| **Raiola Networks** | «Hosting Web (Eres principiante)»: 8,95 €/mes | Verificado (raiolanetworks.com) | No encontrado en la página consultada | España (según la web) | Preguntar por SSH, cron y PHP antes de contratar |
| **Webempresa** | Mini 2,96 €, Medium 5,94 €, Maxi 8,94 €/mes con pago anual | Verificado. Es el precio del primer periodo (descuento del 75 %); la renovación no se indica | No encontrado en la página | A elegir: UE o EE. UU. | Hay que pedir el precio de renovación |
| **SiteGround** (España) | StartUp 2,99 €, GrowBig 5,49 €, GoGeek 8,49 €/mes (12 meses); renovación 15,99 €, 27,99 € y 39,99 € | Verificado | SSH incluido en los planes de WordPress; PHP y cron no encontrados | No verificado | Renovaciones muy por encima del precio de entrada |

| Proveedor | Plan y precio | Estado del precio | PHP / MySQL / SSH / cron | Centro de datos | Comentario |
|---|---|---|---|---|---|
| **Hostinger** (España) | Single 1,49 €, Premium 2,39 €, Unlimited 3,29 €/mes (48 meses); renovación 6,99 €, 9,99 € y 16,99 € | Verificado. IVA no indicado | No encontrados en la página | No verificado | Comparar con su VPS |
| **IONOS** | Precios no verificados (fuente secundaria: 4 €, 6 € y 10 €/mes) | Resultado de búsqueda | SSH/SFTP y PHP 8.1 a 8.3 seleccionable (resultado de búsqueda sobre su página de planes) | No verificado | Hay que ver el precio en su web de España |
| Dinahosting, CDmon | — | No disponibles (la página no se abrió) | — | — | Pendiente |

**Lectura.** Los compartidos cuestan 3 a 9 € al mes, pero la mayoría sube mucho en la renovación. El límite importante no es el precio: es que la API necesita un proceso que siempre esté en marcha (el cron de limpieza y las copias) y no siempre se da en el compartido.

## 8.2 VPS (servidor virtual)

| Proveedor | Plan y precio | Estado | Ubicación en la UE | Comentario |
|---|---|---|---|---|
| **Hetzner Cloud** | CX23 (2 vCPU, 4 GB, 40 GB NVMe): 5,49 € + 0,50 € de IPv4, sin IVA | **Estimación**: fuente secundaria de agosto de 2026. La página oficial no muestra precios. Hay subidas de abril de 2026 y, según una fuente secundaria, de junio de 2026 en las líneas CPX y CCX (de hasta un 176 %) | Falkenstein y Núremberg (Alemania); Helsinki (Finlandia). Verificado | Recomendado: buena relación precio y control; panel y API claros. Comprobar el precio en la consola antes de decidir |
| **DigitalOcean** | Basic 1 GiB, 1 vCPU: 6 $/mes (1.000 GiB de salida). Basic 2 GiB, 1 vCPU: 12 $/mes | Verificado (en dólares) | Ámsterdam, Fráncfort (UE); Londres (Reino Unido, fuera de la UE) | Buena documentación; 1 GiB es poco para la API con PHP y MariaDB |
| **Contabo** | Cloud VPS 4 (4 vCPU, 8 GB, 100 GB SSD): 4,40 €/mes con plan de 24 meses | Verificado. La página indica impuestos incluidos; hay un precio mayor sin etiquetar | UE y Reino Unido | Precio bajo; comprobar condiciones, permanencia y soporte |
| **OVHcloud** | VPS-1 (2 vCore, 4 GB, 40 GB NVMe): desde 4,54 $/mes (pago anual por adelantado, inferido) | Verificado (en dólares) | Ubicaciones europeas no detalladas en la página | Sin España mencionada |
| Clouding.io, Hostinger VPS | — | No disponibles o no verificados | — | Pendiente |

## 8.3 Web estática y CDN (para la web y los cursos)

| Servicio | Plan gratis | Límites verificados | Precio de pago | Comentario |
|---|---|---|---|---|
| **Cloudflare Pages** | Sí | 500 compilaciones al mes; 20.000 archivos por sitio; 25 MiB por archivo; 100 proyectos por cuenta. La página no indica el ancho de banda | — | Recomendado para la web estática. Hay que confirmar el uso comercial en el plan gratis |
| **GitHub Pages** | Sí | 1 GB por sitio; 100 GB al mes y 10 compilaciones por hora (límites blandos) | — | La página no dice si sirve para repositorios privados ni en qué planes: el proyecto es privado; comprobarlo |
| **Netlify** | Sí («Starter»), con sistema de créditos (300) | Salida de 20 créditos por GB; el uso comercial no se menciona | Pro: 20 $/mes por equipo, miembros ilimitados | Alternativa válida; el sistema de créditos complica las cuentas |
| **Bunny.net** | No | CDN de 0,01 $/GB en Europa y Norteamérica; mínimo 1 $ al mes | Según uso | Útil si el tráfico de los cursos crece mucho |

## 8.4 Correo transaccional (códigos de acceso, recuperación y contacto)

| Servicio | Plan gratis | Plan de pago | Notas |
|---|---|---|---|
| **Resend** | 3.000 correos al mes, máximo **100 al día**, sin excedente; retención de registros de 30 días. Verificado | Pro: 20 $/mes con 50.000 correos (35 $ con 100.000); excedente de 0,90 $ por cada 1.000. Scale: desde 90 $/mes con 100.000. Verificado | Empresa estadounidense: hay transferencia fuera de la UE (ver RGPD, capítulo 9). El límite diario de 100 es el que más se nota en un día de lanzamiento |
| **Amazon SES** | No hay nivel gratis permanente (verificado) | 0,10 $ por cada 1.000 correos, pago por uso (verificado) | Muy barato; requiere configurar la verificación de dominio y salir del entorno de pruebas |
| **Brevo** | Resultado de búsqueda: 300 correos al día (no oficial) | Resultado de búsqueda: desde 9 $/mes por 5.000 correos | Empresa francesa; el dato no se ha verificado en su web |
| Postmark, Mailgun, Mailjet, SMTP del hosting | — | — | No consultados hoy |

**Entregabilidad (que los códigos lleguen a la bandeja de entrada).** Hay que configurar tres registros DNS en el dominio: **SPF** (qué servidores pueden enviar en nombre del dominio), **DKIM** (firma del correo, con la clave que da el proveedor) y **DMARC** (qué hacer con los correos que fallan). Los nombres exactos los da cada proveedor. Como base, DMARC empieza en `p=none` para ver los informes, y pasa a `quarantine` cuando todo esté bien configurado. Ejemplo de DMARC:

```
_dmarc.paglot.app.   TXT   "v=DMARC1; p=none; rua=mailto:dmarc@paglot.app; adkim=s; aspf=s"
```

## 8.5 Dominios

| Dominio | Precio | Estado |
|---|---|---|
| `.es` (primer año) | 5,99 € + IVA (OVHcloud) o 9,99 € con promoción de 0,01 € el primer año (Hostinger) | Resultado de búsqueda, de 2026 |
| `.es` (renovación) | 6,99 € + IVA (OVHcloud) | Resultado de búsqueda |
| `.es` (tarifa oficial de Red.es) | 27,59 € al año (documento de 2008) | Histórico; **no vigente** |
| `.com` | No verificado; estimación de 10 a 15 € al año + IVA | Estimación |

El código y los correos usan **paglot.app**. Hay que comprobar que el dominio está comprado a tu nombre y que los registros DNS de la web y del correo están bajo tu control. Si se quiere un dominio `.es`, la renovación ronda los 7 a 10 € al año con IVA (estimación).

### 8.6 Otros servicios

- **Cloudflare Turnstile** (comprobación anti-robots): la página consultada no indica precio ni límites. Según Cloudflare es gratuito, pero hay que comprobarlo antes de contar con él.
- **Certificados HTTPS con Let's Encrypt**: gratuitos (referencia; no consultada hoy).
- **Vigilancia de disponibilidad** (un servicio externo que llama a `/salud` cada minuto): hay planes gratuitos en varios proveedores (no verificado hoy; apartado 9.5).
- **Supabase** (para comparar): plan gratis con 500 MB de base de datos, 50.000 usuarios activos al mes y 5 GB de salida; plan Pro a 25 $/mes con 8 GB, 100.000 usuarios activos al mes y 250 GB de salida (verificado). No figura límite de peticiones por segundo: los planes dicen «Unlimited API requests».

### 8.7 PHP: qué versión elegir

Según fuentes secundarias (php.net no se pudo leer entero):

| Versión | Soporte | Fin de la seguridad |
|---|---|---|
| 8.2 | Solo seguridad | 31-12-2026 |
| 8.3 | Solo seguridad desde noviembre de 2025 | 31-12-2027 |
| 8.4 | Activo hasta el 31-12-2026; seguridad después | 31-12-2028 |
| 8.5 | Activo | Activo hasta el 31-12-2027; seguridad hasta el 31-12-2029 |

**Recomendación.** Instalar **PHP 8.4** (o **8.5** si el sistema operativo ya lo trae) y planear la actualización a 8.5 en 2027. No conviene empezar con 8.3, que ya solo recibe correcciones de seguridad.

### 8.8 Recomendación por escala

**Escenario A: de 0 a 10.000 cuentas.** Un VPS de la UE con Nginx, PHP 8.4 y MariaDB, la web estática en Cloudflare Pages y el correo en Resend.

| Concepto | Proveedor | €/mes sin IVA | Estado |
|---|---|---|---|
| Servidor 2 vCPU, 4 GB, 40 GB NVMe | Hetzner CX23 (Alemania o Finlandia) | 5,99 | Estimación |
| Copias cifradas fuera del servidor | Almacenamiento en la UE (p. ej., Hetzner Storage Box o similar) | 3 a 5 | Estimación; no verificado |
| Dominio `.es` (prorrateado) | — | 0,8 | Estimación |
| Web estática y CDN | Cloudflare Pages | 0 | Verificado (límites) |
| Comprobación anti-robots | Cloudflare Turnstile | 0 | Según Cloudflare; verificar |
| Correo hasta 100 al día | Resend, plan gratis | 0 | Verificado |
| **Subtotal sin IVA** | | **10 a 12** | Estimación |
| Con IVA del 21 % | | **12 a 14** | Estimación |
| Si se pasa de 100 correos al día: Resend Pro | Resend | ≈ 18 (20 $) | Estimación con tipo de cambio orientativo |
| **Total con Resend Pro** | | **≈ 30 a 32 con IVA** | Estimación |

Hay que sumar el tiempo de administración: calcular entre 2 y 4 horas al mes (estimación) para actualizaciones, copias y vigilancia.

**Escenario B: 100.000 cuentas.** Dos servidores de aplicación (4 vCPU y 8 GB cada uno), un servidor de base de datos, un balanceador, Resend Pro para 100.000 correos y copias con más capacidad.

| Concepto | €/mes sin IVA | Estado |
|---|---|---|
| 2 servidores de aplicación (4 vCPU, 8 GB) | 30 a 40 | Estimación (tras las subidas de 2026) |
| 1 servidor de base de datos (4 vCPU, 8 GB) | 15 a 20 | Estimación |
| Balanceador de carga | 5 a 10 | Estimación |
| Resend Pro (100.000 correos al mes, 35 $) | ≈ 31 | Verificado en dólares; conversión estimada |
| Copias y dominio | 6 a 11 | Estimación |
| **Total** | **≈ 85 a 115** | Estimación |

Con Supabase Pro, el equivalente rondaría los **25 $ al mes** del plan base, más los excedentes de salida y de cómputo, que no se han podido verificar (estimación: entre 25 y 60 $).

**Conclusión.** El coste no separa las dos opciones: son del mismo orden. Lo que decide es el **control**: dónde están los datos, quién los trata, qué se puede configurar y no depender de un proveedor concreto. A cambio, el trabajo de administración es real y pesa más que el dinero. Si el mantenimiento no se puede asumir, la alternativa razonable es **quedarse en Supabase con su región comprobada en la UE**, cerrar los agujeros del capítulo 2 y seguir cumpliendo el RGPD con su contrato de encargado.

## 9. Seguridad y operación

### 9.1 HTTPS, cabeceras y entrada única

- **TLS** con certificados de Let's Encrypt (renovación automática con certbot), solo TLS 1.2 y 1.3, y **HSTS** de un año.
- **Una sola puerta**: Nginx sirve `api/public/` y todo va a `index.php`. Nada del código, la configuración ni los registros es accesible desde fuera.
- **Límite de peticiones** por IP en Nginx (`limit_req`, 10 por segundo con ráfagas de 30) y en PHP (`Limites`). Son dos capas distintas: la de Nginx frena el ruido, la de PHP frena la fuerza bruta.
- **Tamaño máximo** del cuerpo: 2 MB (`client_max_body_size`).
- **Cabeceras** en las respuestas de la API: `nosniff`, `Referrer-Policy: no-referrer`, `Cache-Control: no-store`.
- **Web**: si la web pasa a Cloudflare Pages, sus cabeceras se configuran allí. Mientras la CSP de `index.html` no incluya la nueva API, la app no podrá hablar con ella.

`despliegue/nginx-api.conf (Nginx para api.paglot.app)`

```
# Nginx + PHP-FPM para la API. Solo la carpeta public/ es visible; todo va a index.php.
# Va en /etc/nginx/sites-available/api.paglot.app (y enlace en sites-enabled).

limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;

server {
    listen 80;
    server_name api.paglot.app;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name api.paglot.app;

    ssl_certificate     /etc/letsencrypt/live/api.paglot.app/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.paglot.app/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;

    root /var/www/paglot/api/public;
    client_max_body_size 2m;                        # el progreso más grande: 2 MB

    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Content-Type-Options "nosniff" always;

    location ~ /\. { deny all; }                    # nada de archivos ocultos

    location / {
        limit_req zone=api burst=30 nodelay;
        include fastcgi_params;                     # trae REMOTE_ADDR, que aquí ya es la IP real del cliente
        fastcgi_param SCRIPT_FILENAME $document_root/index.php;
        fastcgi_pass unix:/run/php/php8.4-fpm.sock;
    }
}
```

## 9.2 Secretos y acceso al servidor

| Qué | Dónde | Regla |
|---|---|---|
| `PAGLOT_SECRETO` (HMAC) | Variable de entorno o `config/config.php` con permisos 600 | Nunca en Git. Guardarlo en el gestor de contraseñas. Si se pierde, las cuentas quedan inaccesibles |
| Contraseñas de la base (`paglot_app`, `paglot_tareas`) | Igual | Distintas entre sí. Largas y aleatorias |
| Clave SMTP de Resend | Igual | Permiso solo de enviar, limitada al dominio |
| Clave de Turnstile (secreta) | Igual | La de la web es pública; la secreta, no |
| Clave de la copia cifrada (`gpg`) | **Fuera del servidor**, con la pública en el servidor | Sin la privada no hay restauración: guardarla en dos sitios |
| Acceso SSH | VPS | Solo con clave, sin contraseña; usuario no root; `ufw` con solo 22, 80 y 443; `fail2ban` |
| Repositorio | GitHub privado | No subir `config/`, `vendor/`, `buzon/` ni archivos `.env` (añadirlos a `.gitignore`) |

**El material antiguo.** Las carpetas de `Paglot_material_antiguo` contienen secretos reales (`servidor/config.php`, `servidor/local/.claves`, `servidor/local/pruebas-google-claves.json`). Lo prudente: sacarlos de esa carpeta, borrarlos si ya no hacen falta y **cambiar** las contraseñas y claves que contienen, por si alguna vez salieron de tu equipo.

## 9.3 Copias de seguridad (regla 3-2-1)

| Copia | Qué | Dónde | Cuándo |
|---|---|---|---|
| 1. En el servidor | Volcado de la base (cifrado) | `/var/backups/paglot`, 7 días | Cada noche |
| 2. Fuera del servidor | Mismo volcado | Almacenamiento en la UE (otro proveedor distinto del VPS) | Cada noche |
| 3. Mensual | Volcado completo, en un soporte aparte | Disco externo cifrado, en otro lugar | Cada mes |

Hay que **probar la restauración** cada mes en otra máquina. Una copia que no se ha restaurado no se sabe si sirve.

`despliegue/respaldo.sh (copia diaria cifrada)`

```
#!/bin/sh
# Copia diaria de la base (cron, 02:30 UTC). Se cifra ANTES de salir de la máquina.
# Requiere: mariadb-client, gzip, gpg (con la clave pública del responsable) y rclone con un destino en la UE.
set -eu
FECHA=$(date -u +%Y-%m-%d)
DIR=/var/backups/paglot
mkdir -p "$DIR"

mariadb-dump --defaults-extra-file=/etc/paglot/copia.cnf --single-transaction --routines paglot \
  | gzip -9 \
  | gpg --batch --yes --trust-model always --encrypt --recipient "$CLAVE_COPIAS" \
  > "$DIR/paglot-$FECHA.sql.gz.gpg"
```

```
rclone copy "$DIR/paglot-$FECHA.sql.gz.gpg" destino-ue:paglot-copias/
find "$DIR" -name 'paglot-*.sql.gz.gpg' -mtime +7 -delete
echo "Copia $FECHA lista"
```

## 9.4 Actualizaciones

| Qué | Cómo | Frecuencia |
|---|---|---|
| Sistema operativo | `unattended-upgrades` para parches de seguridad | Automático |
| PHP | Seguir el calendario de soporte (apartado 8.7); pasar a la versión siguiente antes de que acabe la seguridad | Anual |
| MariaDB | Versión LTS; parches menores | Trimestral |
| PHPMailer y demás paquetes de Composer | `composer audit` y `composer update` controlado | Trimestral |
| Certificados | certbot con renovación automática | Automático; revisar cada mes |

## 9.5 Vigilancia y registros

- **Disponibilidad.** Un servicio externo que llama a `/api/v1/salud` cada minuto y avisa por correo si falla dos veces seguidas (no verificado hoy qué servicio conviene).
- **Recursos.** Alarma de espacio en disco (la base y las copias crecen) y de memoria.
- **Registros de Nginx.** Contienen IP, que es dato personal. Retención de 14 días, justificada por seguridad, y rotación automática.
- **Registros de PHP.** Solo tipo de error, archivo y línea, nunca datos de la petición (ya es así en `Arranque.php`).
- **Tabla de errores de la app.** Revisar cada semana los errores más repetidos.

## 9.6 Plan de incidentes

1. **Detectar.** Por la vigilancia, por un aviso de usuario o por un registro extraño.
2. **Contener.** Según el caso: cerrar sesiones de forma masiva (`DELETE FROM tokens_dispositivo`), bajar el límite de peticiones en Nginx, poner la API en mantenimiento.
3. **Valorar.** ¿Qué datos? ¿Cuántas personas? ¿Hay riesgo para los derechos y libertades? Anotarlo en el registro de incidentes.
4. **Notificar.** Si hay riesgo, a la **AEPD en 72 horas** desde que se tiene constancia (artículo 33 del RGPD). Si el riesgo es alto, también a las personas afectadas (artículo 34). Aquí hace falta el formulario de la AEPD y un texto corto para los usuarios.
5. **Corregir y contar.** Arreglar la causa y escribir qué pasó, cuándo y qué se hizo. Ese registro es lo que pide la AEPD si pregunta.

## 9.7 RGPD y textos legales

**Registro de actividades (artículo 30).** Borrador para completar:

| Campo | Contenido propuesto |
|---|---|
| Responsable | Nombre, NIF y dirección del titular (hoy `TITULAR` vacío en `js/config.js`) |
| Actividad | Gestión de cuentas y progreso de aprendizaje; informes de errores; formulario de contacto; laboratorio de ideas |
| Finalidad | Prestar el servicio (la cuenta y la sincronización), mantenerlo (errores) y atender consultas |
| Base jurídica | Ejecución del servicio pedido (art. 6.1.b) y consentimiento para el correo opcional y el nombre (art. 6.1.a) |
| Categorías de datos | Correo (en forma de huella y enmascarado), nombre opcional, progreso, actividad, errores anónimos, mensajes de contacto |
| Destinatarios / encargados | Proveedor del VPS (UE), Cloudflare (web, Turnstile), Resend u otro de correo, proveedor de copias |
| Transferencias fuera de la UE | Si se usa Resend o Cloudflare, sí, salvo que tengan un marco de transferencia aplicable (comprobar) |
| Plazos | Ver apartado 4.1 |
| Medidas | Este informe, capítulos 4, 5 y 9 |

**Encargados del tratamiento (artículo 28).** Hay que tener firmado el contrato de encargado de cada proveedor que trate datos por encargo: Hetzner (o el proveedor del VPS), Cloudflare, Resend (o el de correo), el proveedor de copias y, si se queda, **Supabase**. Cada uno publica su DPA; hay que aceptarlo y guardarlo.

**Región de Supabase.** No consta en el código. Mirarla en el panel del proyecto antes de decidir nada: si está fuera de la UE, hay transferencia internacional que documentar.

**Textos a cambiar** (`js/i18n/*.js`, claves `legal.*`, y la página `#/legal`):

1. Quitar la frase de la contraseña cifrada; decir que la cuenta se abre con un número y, si se quiere, con un correo.
2. Decir que los **informes de errores** salen sin cuenta y sin correo, y qué contienen.
3. Decir que las cuentas anónimas se borran a los **7 días sin uso**, y que los informes de errores se guardan **90 días**.
4. Decir que la **dirección IP** la reciben los proveedores (VPS, Cloudflare, correo) para prestar el servicio, y durante cuánto tiempo se guardan los registros.
5. Listar a todos los **encargados** (no solo Supabase).
6. Explicar la **transferencia** fuera de la UE si la hay.
7. Rellenar el **titular** y la fecha de actualización.
8. Explicar que el nombre que se pone en una idea del laboratorio es **público**, y que es opcional.
9. Explicar cómo ejercer los derechos: descargar la copia, borrar la cuenta y pedir acceso o supresión por correo.

**Derechos (capítulos 3 y 5).** Hoy la API cubre el borrado (`DELETE /cuenta`). Falta **exportar** lo que hay en la nube (`GET /cuenta/exportar`), para el derecho de portabilidad. Está en la lista de pendientes del capítulo 5.

**Menores.** LOPDGDD artículo 7: la edad mínima para consentir por sí mismo es 14 años. El aviso lo dice, pero no hay ninguna comprobación: solo es un texto. Si el público infantil crece, conviene revisar esto con un abogado.

## 9.8 Cron

Horas en UTC (la hora del servidor), para que no dependan del horario de verano:

```
# /etc/cron.d/paglot  (formato de /etc/cron.d: el sexto campo es el usuario que ejecuta la tarea)
30 2 * * *  root      /var/www/paglot/scripts/respaldo.sh >> /var/log/paglot-copias.log 2>&1
```

```
30 3 * * *  www-data  php /var/www/paglot/api/tareas/limpieza.php >> /var/log/paglot-limpieza.log 2>&1
```

El envío de correos en cola, si se añade la tabla de reintentos, va en un tercer cron cada cinco minutos (apartado 5.3).

## 10. Checklist paso a paso para Javier

Lo que sigue lo tienes que hacer tú: pagos, cuentas a tu nombre, claves y decisiones. El orden importa. Las fases de la migración están en el capítulo 7.

### 10.1 Antes de nada (esta semana)

- [ ] **Exportar todo lo de Supabase** (capítulo 7.3) y guardarlo cifrado fuera del equipo. Es la primera copia de seguridad, y no se toca.
- [ ] Anotar la **región** del proyecto de Supabase y el número de cuentas (`select count(*) from paglot_cuentas;` en el editor SQL).
- [ ] Revisar en Supabase **Auth → correo y límites** (validez del código, límite de envío) y anotar lo que haya.
- [ ] **Rellenar** `TITULAR` en `js/config.js` y corregir el aviso legal (capítulo 9.7). Es el bloqueo más importante para publicar.
- [ ] **Sacar los secretos** de `Paglot_material_antiguo` (`servidor/config.php`, `servidor/local/.claves`, `servidor/local/pruebas-google-claves.json`) y cambiar lo que contengan.

### 10.2 Dominio, cuentas y servicios

- [ ] Comprobar que **paglot.app** está comprado a tu nombre. Si no, comprarlo (o un `.es` como alternativa).
- [ ] Crear cuenta en **Cloudflare**, añadir el dominio, activar el DNS, y dejar preparadas **Turnstile** (anotar la clave pública y la secreta) y, si se quiere, **Pages**.
- [ ] Crear cuenta en **Hetzner** y dar de alta un VPS en Falkenstein o Helsinki (Ubuntu LTS, 2 vCPU, 4 GB). Antes, mirar el precio en la consola.
- [ ] Crear cuenta en **Resend** y **verificar el dominio** poniendo en el DNS los registros que indique (SPF y DKIM). Añadir DMARC en `p=none`.
- [ ] Elegir y contratar el **almacenamiento de copias** en la UE (otro proveedor distinto del VPS).
- [ ] Aceptar los **contratos de encargado** de cada proveedor y guardarlos (capítulo 9.7).

### 10.3 Servidor

- [ ] En el VPS: usuario no root, acceso SSH solo con clave, `ufw` con 22, 80 y 443, `fail2ban`, `unattended-upgrades`.
- [ ] Instalar **Nginx**, **PHP 8.4-FPM** (con `pdo_mysql`, `mbstring`, `curl`, `openssl`), **MariaDB 11.8 LTS** y **certbot**.
- [ ] Crear el certificado para `api.paglot.app` (registro DNS `A` y `AAAA` apuntando al VPS primero).
- [ ] Copiar `api/` a `/var/www/paglot/api/`, con `vendor/` (Composer) y **sin** `buzon/`.
- [ ] Crear `config/config.php` con los valores reales (o las variables de entorno), permisos 600 y dueño `www-data`.
- [ ] Ejecutar `sql/esquema.sql` y `sql/permisos.sql` (cambiando las contraseñas).
- [ ] Poner los cron del capítulo 9.8 y el script de copias (`scripts/respaldo.sh`) con la clave pública de `gpg`.
- [ ] Probar una **restauración** de copia en otra máquina.

### 10.4 Pruebas

- [ ] Pasar las pruebas del capítulo 6.5 contra el servidor de **pruebas** (no contra producción).
- [ ] Prueba de dos dispositivos (capítulo 6.6).
- [ ] Prueba de carga y de concurrencia básica (capítulo 6.7).
- [ ] Probar la app con **dos móviles y un ordenador** en la API de pruebas.

### 10.5 Migración

- [ ] Fase 0: importar una copia de los datos en el servidor de pruebas y verificar (capítulo 7.6).
- [ ] Fase 1: avisar a los usuarios con una semana de antelación.
- [ ] Fase 2 (día del corte): congelar escrituras en Supabase (los `revoke` del capítulo 7.7).
- [ ] Fase 3: carga final y verificación.
- [ ] Fase 4: publicar la web y el APK con `API_URL` nueva, y la CSP actualizada. Probar en móviles reales.
- [ ] Fase 5: mantener el **adaptador** para las APK antiguas durante meses.
- [ ] Fase 6: Supabase en solo lectura **90 días**; no borrar nada antes.

### 10.6 Cuando todo esté en marcha

- [ ] Revisar cada mes: copias, restauración, actualizaciones, errores de la app y uso del correo.
- [ ] Revisar cada año: versión de PHP, contratos de encargado, precios y el texto legal.
- [ ] Cuando la API supere unas 5.000 cuentas o las copias pasen de 1 GB, repetir el cálculo del capítulo 8.8.

### 10.7 Comandos de referencia

```
# Windows: comprobar que PHP trae las extensiones necesarias
php -m | Select-String -Pattern "pdo_mysql|mbstring|curl|openssl"

# Servidor: comprobar la API y el certificado
curl -s https://api.paglot.app/api/v1/salud
echo | openssl s_client -connect api.paglot.app:443 -servername api.paglot.app 2>/dev/null | openssl x509 -noout -dates
```

---

## Fuentes

Fecha de consulta de todas: **19-10-2026**. «Verificado» indica que la página se abrió y se leyó hoy. «Resultado de búsqueda» indica una fuente secundaria o una página que no se abrió. «Referencia» indica una fuente conocida que **no** se consultó hoy: hay que comprobarla antes de citarla en un documento oficial.

**Proveedores (verificado):**

1. Hetzner Cloud, página principal: https://www.hetzner.com/cloud (sin precios visibles; regiones de Alemania y Finlandia).
2. Raiola Networks: https://www.raiolanetworks.com/ (Hosting Web 8,95 €/mes sin IVA).
3. SiteGround, hosting web: https://www.siteground.es/hosting-web.htm (StartUp, GrowBig, GoGeek).
4. Webempresa: https://www.webempresa.com/ (Mini, Medium, Maxi).
5. Hostinger, hosting web: https://www.hostinger.com/es/hosting-web (redirige a `hostinger.com/es/hosting-web`; planes Single, Premium, Unlimited).
6. OVHcloud, VPS: https://www.ovhcloud.com/es/vps/ (VPS-1 a VPS-4 en dólares).
7. DigitalOcean, Droplets: https://www.digitalocean.com/pricing/droplets (Basic en dólares; regiones de Ámsterdam, Londres y Fráncfort).
8. Contabo, VPS: https://contabo.com/en/vps/ (Cloud VPS 4 a 18).
9. Amazon SES, precios: https://aws.amazon.com/ses/pricing/ (0,10 $ por cada 1.000; sin nivel gratis permanente).
10. GitHub Pages, límites: https://docs.github.com/en/pages/getting-started-with-github-pages/github-pages-limits
11. Bunny.net, precios: https://bunny.net/pricing/ (CDN 0,01 $/GB en Europa y Norteamérica).
12. Netlify, precios: https://www.netlify.com/pricing/ (sistema de créditos; Pro a 20 $/mes por equipo).
13. Cloudflare Pages, límites: https://developers.cloudflare.com/pages/platform/limits/ (500 compilaciones; 20.000 archivos; 25 MiB).
14. Cloudflare Turnstile: https://developers.cloudflare.com/turnstile/ (la página no indica precio ni límites).
15. Supabase, precios: https://supabase.com/pricing (Free 0 $; Pro 25 $/mes; «Unlimited API requests»).
16. Resend, precios: https://resend.com/pricing (Free 3.000/mes y 100/día; Pro 20 $ y 35 $; Scale desde 90 $).

**No disponibles al consultar (error 404 o página no cargada):**

17. IONOS, hosting web: https://www.ionos.es/hosting/hosting-web
18. Dinahosting, hosting web: https://dinahosting.com/hosting-web
19. CDmon, hosting web: https://www.cdmon.com/es/hosting-web
20. Clouding.io, VPS: https://clouding.io/es/vps/
21. Brevo, precios: https://www.brevo.com/pricing/ (la página no mostró datos).

**Resultados de búsqueda** (fuentes secundarias, 19-10-2026):

22. Hetzner, subida de precios de abril de 2026: https://agentdeals.dev/hetzner-pricing-2026
23. Hetzner, comparativa de agosto de 2026 (CX23 y CPX22): https://kimmo.suominen.com/stuff/cpc-2026-08.txt
24. Hetzner, subidas de junio de 2026 en CPX y CCX: https://wz-it.com/blog/hetzner-preiserhoehung-juni-2026-cpx-ccx-alternativen/
25. IONOS, precios de 2026 (guía alemana): https://www.l-iz.de/vpn/erfahrungen/ionos-preise/
26. IONOS, planes con SSH y PHP 8.1 a 8.3: https://www.ionos.com/hosting/web-hosting-plans
27. Brevo, plan gratis de 300 correos al día (comparativa): https://toolquestor.com/es/vs/brevo-vs-turbosmtp
28. PHP, fechas de soporte (secundaria): https://endoflife.ai/article-php-eol.html
29. PHP, página oficial de versiones (solo se vio en el resultado de búsqueda): https://www.php.net/supported-versions.php
30. Dominio `.es`, OVHcloud: https://www.ovhcloud.com/es/domains/tld/es/ (5,99 € + IVA el primer año; 6,99 € + IVA la renovación)
31. Dominio `.es`, Hostinger: https://www.hostinger.com/es/dominio-es (9,99 € al año; 0,01 € el primer año)
32. Dominio `.es`, tarifa histórica de Red.es (27,59 €), noticia: https://www.consumer.es/?p=183106
33. Mullvad, números de cuenta de 16 cifras (blog de hace unos nueve años, posiblemente desfasado): https://mullvad.net/en/blog/mullvads-account-numbers-get-longer-and-safer

**Referencias (no consultadas hoy):**

34. Reglamento (UE) 2016/679 (RGPD): https://eur-lex.europa.eu/eli/reg/2016/679/oj
35. Ley Orgánica 3/2018 (LOPDGDD): https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673
36. Ley 34/2002 (LSSI), artículo 22.2: https://www.boe.es/buscar/act.php?id=BOE-A-2002-13758
37. Agencia Española de Protección de Datos: https://www.aepd.es
38. Have I Been Pwned, API de contraseñas: https://haveibeenpwned.com/API/v3
39. OWASP, guía de almacenamiento de contraseñas: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
40. Cloudflare Turnstile, validación en el servidor: https://developers.cloudflare.com/turnstile/get-started/server-side-validation/
41. PHPMailer: https://github.com/PHPMailer/PHPMailer
42. Let's Encrypt: https://letsencrypt.org/
43. MariaDB, versiones y soporte: https://mariadb.org/
44. Capacitor, configuración del origen en Android: https://capacitorjs.com/docs/config
45. PostgreSQL, privilegios por defecto sobre funciones (`PUBLIC`): https://www.postgresql.org/docs/current/ddl-priv.html

**Archivos del proyecto leídos** (repositorio, sin modificarlos): `js/nube.js`, `js/config.js`, `js/progress.js`, `js/curso.js`, `js/cursos.js`, `js/errores.js`, `js/vistas/perfil.js`, `js/vistas/contacto.js`, `js/vistas/laboratorio.js`, `js/vistas/legal.js`, `js/i18n/es.js`, `index.html`, `_apk/puente.js`, `_apk/capacitor.config.json`, `_apk/copiar.mjs`, `_herramientas/supabase-cuentas.sql`, `_herramientas/supabase-errores-e-ideas.sql`.

**Material antiguo leído** (solo lectura, fuera del proyecto): `Paglot_material_antiguo\Aplicaciones de idiomas desarrolladas - copia\servidor\` (LEEME.md, bd/esquema.sql, bd/permisos.sql, config.ejemplo.php, publico/index.php, src/Api.php, src/Peticion.php, src/Limites.php, src/Seguridad.php, src/Bd.php, src/Progreso.php, tareas/limpieza.php, tareas/correos.php, src/Correo.php). `config.php` se revisó solo por los nombres de sus claves; sus valores no se han leído para este informe.